



2026–2027



جب کوئی قوم فن اور علم
سے عاری ہو جاتی ہے
تو وہ غربت کو دعوت
دیتی ہے اور جب غربت
آتی ہے تو وہ ہزاروں
جرائم کو جنم دیتی ہے۔

“When a nation becomes devoid
of art and learning, it invites
poverty and when poverty comes
it brings in its wake thousands of
crimes.”

– Sir Syed Ahmad Khan

Lab-V

Course Code- CABSXO5P01



B.Sc. (CA) V Semester Lab Manual

DEPARTMENT OF COMPUTER SCIENCE
ALIGARH MUSLIM UNIVERSITY, ALIGARH

2026–2027

Credits

The following lab manual up-gradation committee updated the lab manual:

- ▶ Prof. Arman Rasool Faridi (Chairperson)
- ▶ Prof. Mohammad UbaidUllah Bokhari
- ▶ Prof. Aasim Zafar
- ▶ Prof. Suhel Mustajab
- ▶ Dr. Asif Irshad Khan
- ▶ Dr. Shafiqul Abidin
- ▶ Dr. Faisal Anwar
- ▶ Dr. Mohammad Sajid
- ▶ Dr. Mohammad Nadeem
- ▶ Dr. Faraz Masood

The following committee member originally design the lab manual:

- ▶ Prof. Mohammad Ubaidullah Bokhari
- ▶ Dr. Arman Rasool Faridi
- ▶ Dr. Faisal Anwer
- ▶ Prof. Aasim Zafar (Converner)

Design & Compilation:

- ▶ Dr. Faraz Masood

Revised Edition: July, 2026

Department of Computer Science, A.M.U., Aligarh (U.P.) India

Department of Computer Science

Aligarh Muslim University, Aligarh

B.Sc. (CA) V

Semester Lab Manual

Lab-V (CABSXO-5P01)

Parallel Computing and Cloud Computing

Credit: 2

Periods per Week: 3

Continuous Assessment: 60

Exams: 40

CONTENTS

Week	Laboratory Focus	Page
1	Basic C Programming and Array Operations	5
2	Basic OpenMP Programming and Number Checks	6
3	OpenMP Task Distribution and Mathematical Computation	7
4	Introduction to OpenMP Environment & Tools	8
5	Parallelizing with Threads & Basic Problems	15
6	Data Environment & Synchronization	22
7	Advanced Work-Sharing & Reduction	29
8	Introduction to MPI Environment & Basic Communication	37
9	Collective Communication Operations	47
10	Parallel Numerical Methods & Advanced Sorting (OpenMP Focus)	55
11	Advanced Parallel Sorting (OpenMP & MPI)	61
12	Cloud Security Fundamentals & Basic Web Services	69
13	Cloud Data Analytics & Advanced Security	73
14	Cloud Simulation with CloudSim	77

COURSE TITLE:	Lab-V	COURSE CODE:	CABSXO-5P01
CREDIT:	2	PERIODS PER WEEK:	3
CONTINUOUS ASSESSMENT:	60	EXAMS:	40

COURSE DESCRIPTION

This course provides an in-depth introduction to the fundamental concepts of parallel computing and cloud computing. It covers the architecture and classifications of parallel computers, including Von-Neumann Architecture and Flynn's Taxonomy. Students will learn about performance metrics such as Amdahl's and Gustafson's Laws and explore practical programming techniques using OpenMP and MPI for shared address space and message-passing platforms. Additionally, the course delves into cloud computing, examining cloud architecture, resource management, and practical usage of cloud platforms like OpenNebula and CloudSim. By the end of the course, students will have a comprehensive understanding of both parallel and cloud computing, enabling them to develop efficient programs and manage cloud resources effectively.

CONTENT

The course content includes an introduction to parallel computing, covering Von-Neumann Architecture, Flynn's Classification, and performance measures such as Amdahl's and Gustafson's Laws. It also explores programming techniques for shared address space platforms using OpenMP and message-passing programming with MPI. Students will delve into parallel algorithms for matrix-vector and matrix-matrix multiplication, as well as parallel sorting algorithms like Bitonic Sort and Quick Sort. The cloud computing section covers cloud architecture, resource management, and practical usage of cloud simulators like CloudSim and platforms such as OpenNebula.

OBJECTIVES

This course is designed to help students in:

- ▶ Obtain an understanding of the fundamental concepts of parallel and cloud computing.
- ▶ Learn the classifications and performance metrics of parallel computers.
- ▶ Develop skills to write simple programs using OpenMP and MPI.
- ▶ Understand cloud computing architecture and resource management techniques.
- ▶ Gain knowledge of cloud computing platforms and simulators.

OUTCOMES

After completing this course, the students would be able to:

- ▶ Explain parallel computers and different classifications of computers.
- ▶ Describe limits, costs, and performance metrics of parallelism.
- ▶ Demonstrate and write simple programs using OpenMP compiler directives.
- ▶ Explain cloud computing and its applications.
- ▶ Explain resource management in cloud computing.

RULES AND REGULATIONS

Students are required to strictly adhere to the following rules.

- ▶ The students must complete the weekly activities/assignments well in time (i.e., within the same week).
- ▶ The students must maintain the Lab File of their completed problems in the prescribed format (Appendix-1).
- ▶ The students must get the completed weekly problems checked and signed by the concerned teachers in the Lab in the immediate succeeding week. Failing which the activities/assignments for that week will be treated as incomplete.
- ▶ At least TEN (10) such timely completed and duly signed weekly activities/assignments are compulsory, failing which students will not be allowed to appear in the final Lab Examination.
- ▶ Late submission would not be accepted after the due date.
- ▶ Cooperate, collaborate and explore for the best individual learning outcomes but copying is strictly prohibited.

APPENDIX-I**Template for the Index of Lab File**

WEEK NO.	PROBLEMS WITH DESCRIPTION	PAGE NO.	SIGNATURE OF THE TEACHER WITH DATE
1	1#		
	2#		
	3#		
2	1#		
	2#		
	3#		
3	1#		
	2#		
	3#		

Note: The students should use Header and Footer mentioning their roll no. & name in footer and page no. in header.

WEEK #1**Basic C Programming and Array Operations****OBJECTIVES**

- ▶ To practice basic C programming concepts.
- ▶ To solve simple mathematical and logical problems using C.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Write C programs for checking palindromes, finding roots of quadratic equations, checking leap years, calculating interest, and determining character types.
- ▶ Implement basic algorithms for prime number generation, LCM, HCF, and array operations.

Problems

1. Write a C program to check if the entered number is a palindrome or not.
2. Write a C program to find the roots of a quadratic equation.
3. Write a C program to determine if the entered year is a leap year or not.
4. Write a C program to calculate the compound and simple interest for a given amount.
5. Write a C program to check whether a character is a vowel or consonant.
6. Write a C program to print a list of prime numbers between given ranges.
7. Write a C program to find the LCM and HCF of two entered numbers.
8. Write an interactive program in C to find the largest and smallest number from a given array of size N.
9. Write an interactive program in C to insert a value at a particular location in an array of size N.
10. Write an interactive program in C to search for a particular value in an array of size N.

WEEK #2**Basic OpenMP Programming and Number Checks****OBJECTIVES**

- ▶ To learn basic parallel programming using OpenMP.
- ▶ To convert and merge arrays and implement OpenMP for basic tasks.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Convert decimal numbers to binary, octal, and hexadecimal systems.
- ▶ Merge arrays and implement basic OpenMP programs.
- ▶ Check even, odd, and Armstrong numbers using OpenMP.

PROBLEMS

1. Write an interactive program in C to convert a decimal number to binary, octal, and hexadecimal systems.
2. Write an interactive program in C to merge two arrays of size N1 and N2 into an array of size N1+N2.
3. Write an OpenMP C program to print "Hello World".
4. Write an OpenMP C program to create multiple threads and print "Hello World".
5. Write a C program to check if the given 20 integers are even or odd.
6. Write an OpenMP C program to check if the given 20 integers are even or odd.
7. Write a C program to check if the given 20 integers are Armstrong numbers or not.
8. Write an OpenMP C program to check if the given 20 integers are Armstrong numbers or not.

WEEK #3**OpenMP Task Distribution and Mathematical Computation****OBJECTIVES**

- ▶ To further practice OpenMP programming.
- ▶ To implement parallel computing for simple mathematical problems.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Implement OpenMP programs for checking perfect numbers, distributing tasks, and computing mathematical values.
- ▶ Write OpenMP programs to efficiently distribute work among threads.

PROBLEMS

1. Write a C program to check if the given 20 integers are perfect numbers or not.
2. Write an OpenMP C program to check if the given 20 integers are perfect numbers or not.
3. Write an OpenMP C program to print "Hello World" 30 times.
4. Write an OpenMP C program to print "Hello World" 100 times and distribute the work equally among threads.
5. Write an OpenMP C program to create multiple threads to print "Hello World" and distribute the work randomly among threads.
6. Write a C program to compute the value of pi.
7. Write an OpenMP C program to compute the value of pi.
8. Write an interactive program in C to find the factorial of a given integer.
9. Write an interactive OpenMP C program to compute the factorial of a given integer.

WEEK #4**Introduction to OpenMP Environment & Tools****OBJECTIVES**

To set up the Dev-C or C++ environment for OpenMP development and run basic "Hello World" parallel programs.

Theory/Concepts

- ▶ Introduction to Parallel Computing and its benefits.
- ▶ Shared-Memory Architectures vs. Distributed-Memory Architectures.
- ▶ What is OpenMP? (Open Multi-Processing)
- ▶ Key features of OpenMP (compiler directives, runtime library routines, environment variables).
- ▶ Why Dev-C++? (Lightweight IDE, easy to set up for OpenMP).
- ▶ Compiler Flags for OpenMP (e.g., -fopenmp for GCC/G++).

Lab Setup

1. Download and Install Dev-C++:
 - Go to the official Dev-C++ website or a reliable source like SourceForge.
 - Download the latest stable version of Dev-C++ (preferably with MinGW-w64 included, as it generally has better OpenMP support).
 - Follow the on-screen instructions for installation.
2. Verify GCC/G++ Version:
 - Open Dev-C++.
 - Go to Tools -> Compiler Options.
 - Under the Programs tab, ensure that gcc.exe and g++.exe are pointing to the correct MinGW-w64 binaries.
 - Open a terminal (e.g., Command Prompt or PowerShell) and type gcc –version and g++ –version. You should see a version number (e.g., GCC 8.1.0 or newer).
3. Configure Dev-C++ for OpenMP:
 - Open Dev-C++.
 - Go to Tools -> Compiler Options.

- Under the Compiler tab, select TDM-GCC 4.9.2 64-bit Release or a similar MinGW-w64 profile. If you don't see one, you might need to install a specific MinGW-w64 distribution that supports OpenMP.
- In the Code Generation tab, under Compiler Flags, you need to add the OpenMP flag. For GCC/G++, this is `-fopenmp`.
 - Alternatively, you can add this flag specifically for a project: Project - > Project Options -> Parameters -> Compiler tab.
- Ensure that `-pthread` (for POSIX threads, often implicitly handled by `-fopenmp`) is also present or available if needed, though `-fopenmp` usually handles the necessary linking for OpenMP.

Lab Exercises (Solved Examples)

Exercise 4.1: Hello OpenMP World!

- ▶ Objective: Compile and run a basic OpenMP program.
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }
    return 0;
}
```

- ▶ Tasks:

1. Create a new C/C++ project in Dev-C++.
2. Copy the code into `main.c` (or `main.cpp`).
3. Compile the code with the `-fopenmp` flag.
4. Run the executable.
5. Observe the output. How many "Hello" messages do you see? Why?
6. Try running the program multiple times. Is the order of output always the same? Why or why not?

- ▶ Sample Output:

- ▶ Hello from thread 1
- ▶ Hello from thread 3

- ▶ Hello from thread 0
- ▶ Hello from thread 2 (Order may vary)

Exercise 4.2: Controlling the Number of Threads

- ▶ Objective: Explicitly set the number of threads.
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    omp_set_num_threads(4); // Set the desired number of threads
    #pragma omp parallel
    {
        printf("Hello from thread %d of %d\n", omp_get_thread_num(),
omp_get_num_threads());
    }
    return 0;
}
```

- ▶ Tasks:
1. Modify the previous program to use `omp_set_num_threads()`.
 2. Compile and run. Verify that the output shows the expected number of threads.
 3. Change the `omp_set_num_threads()` value. What happens if you set it to a very large number? What if you set it to 1?
- ▶ Sample Output (with `omp_set_num_threads(3)`):
 - ▶ Hello from thread 0 of 3
 - ▶ Hello from thread 1 of 3
 - ▶ Hello from thread 2 of 3

Exercise 4.3: Getting Max Threads and Processor Count

- ▶ Objective: Understand `omp_get_max_threads()` and `omp_get_num_procs()`.
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    printf("Maximum threads available: %d\n", omp_get_max_threads());
}
```

```
printf("Number of processors: %d\n", omp_get_num_procs());

#pragma omp parallel
{
    printf("Inside parallel region: Thread %d\n", omp_get_thread_num());
}
return 0;
}
```

► Tasks:

1. Implement the code.
2. Run the program. What do `omp_get_max_threads()` and `omp_get_num_procs()` return on your system? How do these values relate to the number of "Hello" messages?

► Sample Output:

- Maximum threads available: 8
- Number of processors: 4
- Inside parallel region: Thread 1
- Inside parallel region: Thread 3
- Inside parallel region: Thread 0
- Inside parallel region: Thread 2 (Values and order may vary based on CPU cores/threads)

Exercise 4.4: Environment Variable OMP_NUM_THREADS

- Objective: Control the number of threads using an environment variable.
- Code: Use the code from Exercise 1.1.
- Tasks:

1. Compile the "Hello OpenMP World!" program without `omp_set_num_threads()`.
2. Before running, set the `OMP_NUM_THREADS` environment variable (e.g., in Windows Command Prompt: set `OMP_NUM_THREADS=5`).
3. Run the compiled program from the command prompt. Observe the number of threads.
4. Change `OMP_NUM_THREADS` to a different value (e.g., 2). Rerun and observe.
5. Question: What is the precedence order if `omp_set_num_threads()` is used AND `OMP_NUM_THREADS` is set? (Experiment to find out!)

► Sample Output (after set `OMP_NUM_THREADS=5`):

- Hello from thread 0
- Hello from thread 1
- Hello from thread 2

- ▶ Hello from thread 3
- ▶ Hello from thread 4

Lab Exercises (Unsolved Problems)

Problem 4.1: Thread ID Summation

- ▶ **Description:** Write an OpenMP program where each thread adds its thread ID to a shared global variable. Display the final sum.
- ▶ **Hint:** You will need to consider how to correctly sum a shared variable across multiple threads. Don't worry about perfect correctness for now; just observe the behavior.
- ▶ **Expected Output Consideration:** The sum might not be the theoretically correct sum of all thread IDs if not synchronized. This is an intended observation for this week.
- ▶ **Sample Run** (if 4 threads were used and sum was $0+1+2+3=6$):
 - ▶ Thread 0 adding...
 - ▶ Thread 1 adding...
 - ▶ Thread 2 adding...
 - ▶ Thread 3 adding...
- ▶ **Final Sum:** 6 (Note: The actual Final Sum might be incorrect due to race condition. Explain why.)

Problem 4.2: Thread-Specific Messages with Number of Processors

- ▶ **Description:** Write an OpenMP program that prints a unique message from each thread, similar to "Hello from thread X", but also includes the total number of physical processors detected by OpenMP. Ensure the message clearly states both.
- ▶ **Requirements:**
 - Use `omp_get_thread_num()`.
 - Use `omp_get_num_procs()`.
 - The output should be within the parallel region.
- ▶ **Sample Output:**
 - ▶ Thread 0 is running on a system with 4 processors.
 - ▶ Thread 1 is running on a system with 4 processors.
 - ▶ Thread 2 is running on a system with 4 processors.
 - ▶ Thread 3 is running on a system with 4 processors. (Number of processors and order may vary)

Problem 4.3: Dynamic Thread Count by User Input

- ▶ **Description:** Write an OpenMP program that asks the user for the desired number of threads at runtime. Then, create a parallel region using the user-specified number of threads and have each thread print its ID.
- ▶ **Requirements:**
 - Use scanf (or equivalent) to get user input for the number of threads.
 - Use omp_set_num_threads() with the user's input.
- ▶ **Sample Input/Output:**
 - Enter desired number of threads: 3
 - Hello from thread 0
 - Hello from thread 1
 - Hello from thread 2

Problem 4.4: Time a Small Parallel Block

- ▶ **Description:** Write an OpenMP program that measures the execution time of a very small loop (e.g., iterating 1000 times) both inside and outside a parallel region. Compare the timings.
- ▶ **Requirements:**
 - Use omp_get_wtime().
 - Include a simple for loop inside a #pragma omp parallel block.
 - Include the same simple for loop outside any parallel region.
 - Print the time taken for both.
- ▶ **Discussion Point:** Which one is faster and why? (Consider overheads)
- ▶ **Sample Output:**
 - ▶ Time for parallel block: 0.000123 seconds
 - ▶ Time for serial block: 0.000045 seconds (Times will vary; parallel might be slower for very small tasks)

Problem 4.5: Verify OMP_NUM_THREADS Precedence

- ▶ **Description:** Create an OpenMP program where you explicitly set omp_set_num_threads(3) inside main(). Then, compile the program. Before running, set the environment variable OMP_NUM_THREADS=5. Run the program and observe how many threads are created. Based on the output, conclude which method has higher precedence.

- ▶ Requirements:
 - Code must contain `omp_set_num_threads(3)`.
 - You must externally set `OMP_NUM_THREADS=5` before running.
- ▶ Expected Output: The output should reveal whether 3 or 5 threads are active. Explain your observation.
- ▶ Sample Output: (Based on observation)
- ▶ Hello from thread 0
- ▶ Hello from thread 1
- ▶ Hello from thread 2 (This implies `omp_set_num_threads()` has higher precedence)

Problem 4.6: Count Active Threads in Parallel Region

- ▶ Description: Write an OpenMP program that enters a parallel region. Inside the parallel region, each thread should print "I am thread X of Y active threads". Use `omp_get_num_threads()` to determine Y.
- ▶ Requirements:
 - Use `#pragma omp parallel`.
 - Use `omp_get_thread_num()` and `omp_get_num_threads()`.
- ▶ Sample Output (with default threads):
 - I am thread 0 of 4 active threads
 - I am thread 1 of 4 active threads
 - I am thread 2 of 4 active threads
 - I am thread 3 of 4 active threads

WEEK #5**Parallelizing with Threads & Basic Problems**

Objective: To understand how to parallelize code using OpenMP threads, compare serial and parallel execution, and use fundamental OpenMP constructs like parallel for.

Theory/Concepts

- ▶ `#pragma omp parallel`: The fundamental construct to define a parallel region.
- ▶ `omp_get_thread_num()`: Returns the thread ID (0 for master).
- ▶ `omp_get_num_threads()`: Returns the total number of threads in the current team.
- ▶ `#pragma omp parallel for`: A work-sharing construct that parallelizes a for loop.
- ▶ Serial vs. Parallel Execution: Understanding speedup and overhead.
- ▶ `num_threads()` clause: Setting the number of threads for a specific parallel region.

Lab Exercises (Solved Examples)**Exercise 5.1: Simple Loop Parallelization with `#pragma omp parallel for`**

- ▶ Objective: Parallelize a simple loop and observe thread distribution.
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i;
    printf("Serial part of the program.\n");

    #pragma omp parallel for
    for (i = 0; i < 10; i++) {
        printf("Iteration %d from thread %d\n", i, omp_get_thread_num());
    }

    printf("Back to serial part.\n");
    return 0;
}
```

- ▶ Tasks:

1. Implement and run the code.
2. Observe how iterations are distributed among threads. Is the distribution always the same?
3. Change the upper limit of the loop (e.g., to 20, 50). What do you notice about the thread assignments?

► Sample Output (order may vary):

1. Serial part of the program.
2. Iteration 0 from thread 0
3. Iteration 1 from thread 0
4. Iteration 2 from thread 0
5. Iteration 3 from thread 0
6. Iteration 4 from thread 1
7. Iteration 5 from thread 1
8. Iteration 6 from thread 2
9. Iteration 7 from thread 2
10. Iteration 8 from thread 3
11. Iteration 9 from thread 3
12. Back to serial part.

Exercise 5.2: Comparing Serial and Parallel Summation (with Race Condition)

- Objective: Calculate the sum of a large array in serial and parallel, comparing execution times and observing race conditions.
- Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#define ARRAY_SIZE 1000000
int main() {
    long long *arr = (long long *)malloc(sizeof(long long) * ARRAY_SIZE);
    long long serial_sum = 0;
    long long parallel_sum = 0;
    double start_time, end_time;
    int i;
    // Initialize array
    for (i = 0; i < ARRAY_SIZE; i++) {
        arr[i] = i + 1;
    }
    // --- Serial Summation ---
```

```
start_time = omp_get_wtime();
for (i = 0; i < ARRAY_SIZE; i++) {
    serial_sum += arr[i];
}
end_time = omp_get_wtime();
printf("Serial Sum: %lld\n", serial_sum);
printf("Serial Time: %f seconds\n", end_time - start_time);

// --- Parallel Summation (with intended race condition) ---
start_time = omp_get_wtime();
#pragma omp parallel for
for (i = 0; i < ARRAY_SIZE; i++) {
    parallel_sum += arr[i]; // Race condition here!
}
end_time = omp_get_wtime();
printf("Parallel Sum: %lld (might be incorrect due to race condition)\n",
parallel_sum);
printf("Parallel Time: %f seconds\n", end_time - start_time);

free(arr);
return 0;
}
```

► Tasks:

1. Implement and run the code.
2. Compare the serial and parallel execution times. Do you see a speedup?
3. Crucial Observation: Notice that the `parallel_sum` might be incorrect due to a race condition. Discuss why this happens. (This sets the stage for reduction clause in future labs).

Sample Output:

- Serial Sum: 50000000500000
- Serial Time: 0.123456 seconds
- Parallel Sum: 499999876543210 (incorrect due to race condition)
- Parallel Time: 0.056789 seconds

Exercise 5.3: num_threads Clause with parallel for

- Objective: Explicitly control the number of threads for a specific parallel loop.
- Code:

```
#include <stdio.h>
#include <omp.h>
```

```
int main() {
    int i;

    printf("Running with default threads:\n");
    #pragma omp parallel for
    for (i = 0; i < 5; i++) {
        printf("Default: Iteration %d from thread %d\n", i, omp_get_thread_num());
    }

    printf("\nRunning with 2 threads:\n");
    #pragma omp parallel for num_threads(2)
    for (i = 0; i < 5; i++) {
        printf("2 Threads: Iteration %d from thread %d\n", i, omp_get_thread_num());
    }

    printf("\nRunning with 8 threads (if available):\n");
    #pragma omp parallel for num_threads(8)
    for (i = 0; i < 5; i++) {
        printf("8 Threads: Iteration %d from thread %d\n", i, omp_get_thread_num());
    }

    return 0;
}
```

► Tasks:

1. Implement and run the code.
2. Observe how the number of threads changes for each parallel for block.
3. What happens if you request more threads than available hardware threads? Sample Output (with 4 physical cores): Running with default threads:

- Default: Iteration 0 from thread 0
- Default: Iteration 1 from thread 0
- Default: Iteration 2 from thread 1
- Default: Iteration 3 from thread 2
- Default: Iteration 4 from thread 3

Running with 2 threads:

- 2 Threads: Iteration 0 from thread 0
- 2 Threads: Iteration 1 from thread 0
- 2 Threads: Iteration 2 from thread 1
- 2 Threads: Iteration 3 from thread 1

- ▶ 2 Threads: Iteration 4 from thread 1

Running with 8 threads (if available):

- ▶ 8 Threads: Iteration 0 from thread 0
- ▶ 8 Threads: Iteration 1 from thread 1
- ▶ 8 Threads: Iteration 2 from thread 2
- ▶ 8 Threads: Iteration 3 from thread 3
- ▶ 8 Threads: Iteration 4 from thread 4

Lab Exercises (Unsolved Problems)

Problem 5.1: Array Initialization - Serial vs. Parallel

- ▶ **Description:** Initialize a large integer array (e.g., `int arr[1000000];`) with values such that `arr[i] = i * 2;`. Implement this initialization both serially and in parallel using `#pragma omp parallel for`. Measure and compare the execution times.
- ▶ **Requirements:**
 - Use `omp_get_wtime()` for timing.
 - Print the time taken for both serial and parallel initialization.
 - Verify a few elements of the array after parallel initialization to ensure correctness.
- ▶ **Sample Output:**
 - Serial Array Initialization Time: 0.005123 seconds
 - Parallel Array Initialization Time: 0.001567 seconds
 - Verified `arr[0]=0`, `arr[100000]=200000`, `arr[999999]=1999998`

Problem 5.2: Element-wise Array Addition

- ▶ **Description:** Given two large arrays, A and B (e.g., `double A[500000], B[500000], C[500000];`), initialize them with arbitrary values. Then, compute `C[i] = A[i] + B[i]` for all i. Implement this calculation serially and in parallel using `#pragma omp parallel for`. Compare their execution times.
- ▶ **Requirements:**
 - Initialize A and B (e.g., `A[i] = i`, `B[i] = i * 0.5`).
 - Compute C serially and time it.
 - Compute C in parallel and time it.
 - Ensure i is correctly handled as a loop variable.
- ▶ **Sample Output:**

- Serial Element-wise Addition Time: 0.004567 seconds
- Parallel Element-wise Addition Time: 0.001234 seconds

Problem 5.3: Finding Minimum Value in an Array

- ▶ **Description:** Find the minimum value in a large integer array (e.g., `int arr[1000000];`) using parallel processing. Initialize the array with random numbers. Do not use the reduction clause yet. This problem is intended to make you think about how to manage shared state (the `min_value` variable) without advanced OpenMP features.
- ▶ **Hint:** You will likely encounter a race condition. A simple if condition might not be enough. Just observe the (potentially incorrect) result and discuss why.
- ▶ **Sample Output:**
- ▶ **Array Min Value (Parallel, potentially incorrect):** 5 (Actual minimum might be 1, but parallel might report 5 due to race condition.)

Problem 5.4: Basic Vector Dot Product

- ▶ **Description:** Calculate the dot product of two vectors, $A \cdot B = \sum_{i=0}^{N-1} A_i B_i$. Use arrays of size $N = 1000000$. Implement both serial and parallel versions. Similar to the summation problem, expect a race condition if not handled carefully.
- ▶ **Requirements:**
 - Initialize vectors A and B.
 - Calculate dot product serially and time it.
 - Calculate dot product in parallel using `#pragma omp parallel for`.
 - Acknowledge the race condition on the sum variable.
- ▶ **Sample Output:**
- ▶ **Serial Dot Product:** 333333335000000.000000
- ▶ **Serial Time:** 0.012345 seconds
- ▶ **Parallel Dot Product (Race Condition):** 333333334999999.000000 (might be incorrect)
- ▶ **Parallel Time:** 0.005678 seconds

Problem 5.5: Counting Even Numbers in an Array

- ▶ **Description:** Count the number of even numbers in a large integer array (e.g., `int arr[2000000];`). Initialize the array with random integers. Implement this serially and in parallel. Focus on managing the shared `even_count` variable.
- ▶ **Hint:** This is another scenario where direct `even_count++` in parallel will lead to a race

condition. Note the incorrect result.

- ▶ Sample Output:
 - Serial Even Count: 999876
 - Serial Time: 0.008765 seconds
 - Parallel Even Count (Race Condition): 999870
 - Parallel Time: 0.003456 seconds

Problem 5.6: Array Element Modification Based on Thread ID

- ▶ **Description:** Create an integer array `arr[20]`. In a parallel region, have each thread modify a specific set of elements based on its `thread_id`. For example, thread 0 modifies `arr[0]`, `arr[1]`, thread 1 modifies `arr[2]`, `arr[3]`, etc. Each thread should set its elements to its own `thread_id`. Then, print the final array from the master thread after the parallel region.
- ▶ **Requirements:**
 - Use `#pragma omp parallel`.
 - Manually determine the range for each thread based on `omp_get_thread_num()` and `omp_get_num_threads()`.
 - Use `omp_set_num_threads(4)` for this specific problem.
- ▶ **Sample Output (with 4 threads):**
- ▶ **Final Array:**
- ▶ 00112233001122330011 (Note: The example output assumes simple modulo distribution or wrap-around if the array size isn't a multiple of `num_threads * elements_per_thread`. Students should aim for non-overlapping, contiguous chunks for simplicity in this problem.)

WEEK #6**Data Environment & Synchronization**

Objective: To understand how data is shared and managed in OpenMP, and to implement basic synchronization constructs to prevent race conditions.

Theory/Concepts

- ▶ Data Sharing Attributes: shared, private, firstprivate, lastprivate, default.
- ▶ private clause: Each thread gets its own copy.
- ▶ firstprivate clause: Each thread gets its own copy, initialized with the master thread's value before the parallel region.
- ▶ lastprivate clause: The value of the variable from the thread that executes the sequentially last iteration is copied back to the original variable in the master thread.
- ▶ Synchronization:
 - `#pragma omp critical`: Ensures that only one thread can execute a block of code at a time.
 - `#pragma omp atomic`: Ensures that a specific memory update (like an increment/decrement) is performed atomically.
 - `#pragma omp barrier`: All threads wait at this point until all threads have reached it.
 - `omp_lock_t` and `omp_set_lock`, `omp_unset_lock`, `omp_init_lock`, `omp_destroy_lock`: Low-level lock functions.

Lab Exercises (Solved Examples)**Exercise 6.1: Exploring shared, private, and default**

- ▶ Objective: Understand the default data sharing and explicitly use shared and private.
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int global_var = 100; // Global variable is shared by default

int main() {
    int i = 10; // Shared by default within parallel region if not specified
    int j = 20; // Shared by default
```

```

    int k = 30; // Shared by default

    #pragma omp parallel default(none) shared(i, global_var) private(j) // k is
    implicitly shared
    if not specified, but here default(none) forces it
    {
        int thread_id = omp_get_thread_num();
        printf("Thread %d: i (shared) = %d, global_var (shared) = %d\n", thread_id,
i,
        global_var);
        printf("Thread %d: j (private) = %d (before modification)\n", thread_id, j)
;

        j = thread_id * 100; // Each thread modifies its private copy of j
        printf("Thread %d: j (private) = %d (after modification)\n", thread_id,
j);

        // If a variable like 'k' was used without being declared private or
        shared,
        // with default(none), it would result in a compilation error.
        // For simplicity, we stick to i, j, global_var for this example.
    }

    printf("\nAfter parallel region:\n");
    printf("Main thread: i = %d, global_var = %d\n", i, global_var);
    // printf("Main thread: j = %d (value undefined after parallel region for
    private j)\n", j); //
    This would be problematic

    return 0;
}

```

► Tasks:

1. Implement and run the code.
2. Observe the values of `i`, `global_var`, and `j`. Explain the behavior of each.
3. What happens if you remove `shared(i, global_var)` or `private(j)` with `default(none)`?
4. Discuss the implications of `default(none)`.

Sample Output:

- Thread 0: i (shared) = 10, global_var (shared) = 100
- Thread 0: j (private) = 20 (before modification)
- Thread 0: j (private) = 0 (after modification)
- Thread 1: i (shared) = 10, global_var (shared) = 100

- Thread 1: j (private) = 20 (before modification)
- Thread 1: j (private) = 100 (after modification)
- Thread 2: i (shared) = 10, global_var (shared) = 100
- Thread 2: j (private) = 20 (before modification)
- Thread 2: j (private) = 200 (after modification)
- Thread 3: i (shared) = 10, global_var (shared) = 100
- Thread 3: j (private) = 20 (before modification)
- Thread 3: j (private) = 300 (after modification)
- After parallel region:
- Main thread: i = 10, global_var = 100

Exercise 6.2: firstprivate and lastprivate

- ▶ Objective: Understand how initial and final values are handled for private variables.
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int x = 10;
    int y = 20;
    int z = 0; // Will be used with lastprivate, initialized here for clarity

    printf("Before parallel region: x = %d, y = %d, z = %d\n", x, y, z);

    #pragma omp parallel for firstprivate(x) lastprivate(y, z)
    for (int i = 0; i < 5; i++) {
        int thread_id = omp_get_thread_num();
        printf("Thread %d, Iteration %d: x (firstprivate) = %d\n", thread_id, i, x);
        x = x + 1; // Each thread increments its own copy of x
        y = i * 10; // Each thread sets its own copy of y
        z = i;      // Each thread sets its own copy of z; last one wins for
lastprivate
        printf("Thread %d, Iteration %d: x (modified) = %d, y (modified) = %d, z (
modified) =
        %d\n", thread_id, i, x, y, z);
    }

    printf("\nAfter parallel region: x = %d (unchanged), y = %d (lastprivate), z
= %d
```

```
(lastprivate)\n", x, y, z);

    return 0;
}
```

► Tasks:

1. Implement and run the code.
2. Observe the values of x, y, and z before and after the parallel region.
3. Explain why x remains its original value after the parallel region, and y and z take on a specific value.
 - Sample Output (assuming 4 threads, order of iterations varies):
 - Before parallel region: x = 10, y = 20, z = 0
 - Thread 0, Iteration 0: x (firstprivate) = 10
 - Thread 0, Iteration 0: x (modified) = 11, y (modified) = 0, z (modified) = 0
 - Thread 1, Iteration 1: x (firstprivate) = 10
 - Thread 1, Iteration 1: x (modified) = 11, y (modified) = 10, z (modified) = 1
 - Thread 2, Iteration 2: x (firstprivate) = 10
 - Thread 2, Iteration 2: x (modified) = 11, y (modified) = 20, z (modified) = 2
 - Thread 3, Iteration 3: x (firstprivate) = 10
 - Thread 3, Iteration 3: x (modified) = 11, y (modified) = 30, z (modified) = 3
 - Thread 0, Iteration 4: x (firstprivate) = 10
 - Thread 0, Iteration 4: x (modified) = 11, y (modified) = 40, z (modified) = 4 •
 - After parallel region: x = 10 (unchanged), y = 40 (lastprivate), z = 4 (lastprivate)

Lab Exercises (Unsolved Problems)

Problem 6.1: Correct Parallel Summation using #pragma omp critical

► **Description:** Take the parallel array summation problem from Week 2 (Problem 2.2). Correct the race condition on the parallel_sum variable using #pragma omp critical. Measure and compare the execution times with the serial version and the previous (incorrect) parallel version.

► Requirements:

- The sum must be correct.

Use #pragma omp critical to protect the sum update. ○

► Sample Output:

- Serial Sum: 5000000050000000

- Serial Time: 0.123456 seconds
- Parallel Sum (Critical): 5000000050000000
- Parallel Time (Critical): 0.087654 seconds

Problem 6.2: Correct Parallel Summation using #pragma omp atomic

► **Description:** Take the parallel array summation problem from Week 2 (Problem 2.2) again. Correct the race condition on the parallel_sum variable using #pragma omp atomic. Measure and compare the execution times with the critical version.

► **Requirements:**

- The sum must be correct.

Use #pragma omp atomic to protect the sum update. o

► **Sample Output:**

► Serial Sum: 5000000050000000

► Serial Time: 0.123456 seconds

► Parallel Sum (Atomic): 5000000050000000

► Parallel Time (Atomic): 0.067890 seconds (Note: Atomic is often faster than critical for simple operations).

Problem 6.3: Finding Max Value with omp_lock_t

► **Description:** Find the maximum value in a large array (e.g., int arr[1000000]; initialized with random values) using omp_lock_t for synchronization. Compare its performance to a serial version.

► **Requirements:**

- Initialize an omp_lock_t variable.
- Use omp_set_lock() and omp_unset_lock() to protect the update of the max_value.
- Destroy the lock at the end.

► **Sample Output:**

► Max Value (Serial): 999987

► Serial Time: 0.007890 seconds

► Max Value (Locked Parallel): 999987

► Parallel Time (Locked Parallel): 0.009876 seconds (Note: Locking can introduce significant overhead, making it slower than serial for simple operations).

Problem 6.4: Data Exchange using omp_barrier

- ▶ **Description:** Have each thread in a parallel region calculate a thread-specific value (e.g., `thread_id * 10`). After all threads have calculated their value, use `#pragma omp barrier` to ensure all threads wait. Then, after the barrier, have each thread print "Thread X's value is Y, and total active threads are Z". This simulates a scenario where threads need to wait for others before proceeding with a task that relies on collective completion.
- ▶ **Requirements:**
 - Use `omp_get_thread_num()` and `omp_get_num_threads()`.
 - Use `#pragma omp barrier`.
- ▶ **Sample Output:**
 - Thread 0 calculated value 0.
 - Thread 1 calculated value 10.
 - Thread 2 calculated value 20.
 - Thread 3 calculated value 30.
 - Thread 0's value is 0, and total active threads are 4.
 - Thread 1's value is 10, and total active threads are 4.
 - Thread 2's value is 20, and total active threads are 4.
 - Thread 3's value is 30, and total active threads are 4.

Problem 6.5: Shared Counter with firstprivate Loop Variable

- ▶ **Description:** Declare a shared integer variable counter initialized to 0. Create a parallel region with a loop that iterates 100 times. Each thread should increment counter inside the loop. The loop variable `i` should be `firstprivate`. Analyze the final value of counter and explain why it behaves that way.
- ▶ **Requirements:** oUse `shared(counter)` and `firstprivate(i)`.
 - Acknowledge the race condition on counter.
- ▶ **Sample Output (with 4 threads):**
- ▶ Thread 0: `i=0`, counter modified.
- ▶ ...
- ▶ Thread 3: `i=0`, counter modified.
- ▶ ...
- ▶ **Final counter value (incorrect):** 25 (Final counter value will be less than 100, possibly related to number of threads or how many ++ operations successfully complete.)

Problem 6.6: Counting Occurrences of a Character in a String Array

► **Description:** Given an array of strings (e.g., `char* words[] = {"apple", "banana", "orange", "grape", "kiwi", "melon", "apricot"};`), count the total occurrences of a specific character (e.g., 'a') across all strings. Implement this in parallel using `#pragma omp parallel for` and `#pragma omp atomic` for the shared counter.

► **Requirements:**

- Iterate through the array of strings in parallel.
- For each string, iterate through its characters. Use `#pragma omp atomic` to safely increment the shared count.

► **Sample Input:**

```
char* words[] = {"apple", "banana", "orange", "grape", "kiwi", "melon", "apricot"}; char target_char = 'a';
```

► **Sample Output:**

- Total 'a' occurrences: 6

.

WEEK #7**Advanced Work-Sharing & Reduction**

Objective: To learn about more advanced work-sharing constructs and to efficiently handle data aggregation (reductions) in parallel.

Theory/Concepts

- ▶ **#pragma omp sections:** Divides a block of code into sections, each executed by a single thread.
- ▶ **#pragma omp single:** Specifies that a block of code is executed by only one thread.
- ▶ **#pragma omp master:** Specifies that a block of code is executed by the master thread only.
- ▶ **#pragma omp ordered:** Ensures that a block of code within a parallel loop is executed in the order of loop iterations.
- ▶ **reduction clause:** A powerful and efficient way to perform parallel data aggregation (sum, min, max, product, etc.) safely and correctly.
 - `reduction(operator:list)`

Lab Exercises (Solved Examples)**Exercise 7.1: Using #pragma omp sections**

- ▶ **Objective:** Execute different code blocks concurrently using sections.
- ▶ **Code:**

```
#include <stdio.h>
#include <omp.h>
#include <unistd.h> // For sleep, on Windows use <windows.h> and Sleep()

void task_a() {
    printf("Task A executed by thread %d\n", omp_get_thread_num());
    usleep(1000 * 500); // 0.5 seconds
}

void task_b() {
    printf("Task B executed by thread %d\n", omp_get_thread_num());
    usleep(1000 * 1000); // 1 second
}
```

```
void task_c() {
    printf("Task C executed by thread %d\n", omp_get_thread_num());
    usleep(1000 * 200); // 0.2 seconds
}
int main() {
    printf("Starting sectioned tasks...\n");

    #pragma omp parallel sections
    {
        #pragma omp section
        {
            task_a();
        }

        #pragma omp section
        {
            task_b();
        }

        #pragma omp section
        {
            task_c();
        }
    }
    printf("All sectioned tasks completed.\n");
    return 0;
}
```

► Tasks:

1. Implement and run the code.
2. Observe which thread executes each task. Do they always run in the same order?
3. How many threads are involved in executing the sections? (Hint: The default number of threads in the parallel region).
 - Sample Output (order of tasks completion may vary):
 - Starting sectioned tasks...
 - Task A executed by thread 0
 - Task C executed by thread 1
 - Task B executed by thread 2
 - All sectioned tasks completed.

Exercise 7.2: Using #pragma omp single

- ▶ Objective: Execute a block of code by only one arbitrary thread.
- ▶ Code: C

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel num_threads(4)
    {
        printf("Thread %d: Before single block.\n", omp_get_thread_num());

        #pragma omp single
        {
            printf("Thread %d: ONLY I execute this block.\n", omp_get_thread_num
());
        }

        printf("Thread %d: After single block.\n", omp_get_thread_num());
    }
    return 0;
}
```

- ▶ Tasks:

1. Implement and run the code.
2. Which thread executes the single block? Is it always the same thread?
3. What is the difference between single and master? (Discuss the implicit barrier after single).

- ▶ Sample Output:

- ▶ Thread 0: Before single block.
- ▶ Thread 1: Before single block.
- ▶ Thread 2: Before single block.
- ▶ Thread 3: Before single block.
- ▶ Thread 0: ONLY I execute this block.
- ▶ Thread 0: After single block.
- ▶ Thread 1: After single block.
- ▶ Thread 2: After single block.
- ▶ Thread 3: After single block.

Exercise 7.3: Using #pragma omp master

- ▶ Objective: Execute a block of code by only the master thread (thread 0).
- ▶ Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel num_threads(4)
    {
        printf("Thread %d: Before master block.\n", omp_get_thread_num());

        #pragma omp master
        {
            printf("Thread %d: I am the MASTER thread, and ONLY I execute this block.\n",
",
omp_get_thread_num());
        }
        printf("Thread %d: After master block.\n", omp_get_thread_num());
    }
    return 0;
}
```

- ▶ Tasks:

1. Implement and run the code.
2. Verify that only thread 0 executes the master block.
3. Compare and contrast single and master. (No implicit barrier after master).

- ▶ Sample Output:

- ▶ Thread 0: Before master block.
- ▶ Thread 1: Before master block.
- ▶ Thread 2: Before master block.
- ▶ Thread 3: Before master block.
- ▶ Thread 0: I am the MASTER thread, and ONLY I execute this block.
- ▶ Thread 1: After master block.
- ▶ Thread 0: After master block.
- ▶ Thread 2: After master block.
- ▶ Thread 3: After master block. (Note the master thread printing "After master block" before other threads due to no implicit barrier.)

Lab Exercises (Unsolved Problems)

Problem 7.1: Correct Parallel Summation with reduction

- ▶ Description: Take the parallel array summation problem from Week 2 (Problem 2.2) and Week 3 (Problems 3.1 & 3.2). Now, use the reduction clause to correctly sum the array elements. Compare its performance with the serial, critical, and atomic versions.
- ▶ Requirements:
 - Use reduction(+:parallel_sum).
 - Compare timings.
- ▶ Sample Output:
 - Serial Sum: 5000000050000000
 - Serial Time: 0.123456 seconds
 - Parallel Sum (Critical): 5000000050000000
 - Parallel Time (Critical): 0.087654 seconds
 - Parallel Sum (Atomic): 5000000050000000
 - Parallel Time (Atomic): 0.067890 seconds
 - Parallel Sum (Reduction): 5000000050000000
 - Parallel Time (Reduction): 0.034567 seconds

Problem 7.2: Finding Max and Min in an Array using reduction

- ▶ Description: Write an OpenMP program to find both the maximum and minimum values in a large integer array (e.g., `int arr[1000000]`; with random values). Use the reduction clause for both operations within a single parallel region if possible, or separate parallel regions if necessary.
- ▶ Requirements:
 - Use reduction(max:max_val) and reduction(min:min_val).
 - Initialize max_val to INT_MIN and min_val to INT_MAX (from `<limits.h>`).
- ▶ Sample Output:
 - Max Value: 999999
 - Min Value: 0

Problem 7.3: Dot Product with reduction

- ▶ Description: Revisit the dot product problem from Week 2 (Problem 2.4). Correctly calcu-

late the dot product of two large vectors using the reduction clause. Compare its performance against the previous (incorrect) parallel version and the serial version.

- ▶ Requirements:
 - Use reduction(+:dot_product).
- ▶ Sample Output:
 - Serial Dot Product: 333333335000000.000000
 - Serial Time: 0.012345 seconds
 - Parallel Dot Product (Reduction): 333333335000000.000000
 - Parallel Time (Reduction): 0.003456 seconds

Problem 7.4: Averaging Array Elements with reduction

- ▶ Description: Calculate the average of elements in a large floating-point array (e.g., float arr[1000000];). Use reduction to sum all elements, then calculate the average.
- ▶ Requirements:
 - Initialize the array with random float values.
 - Use reduction(+:total_sum).
 - Calculate average = total_sum / ARRAY_SIZE.
- ▶ Sample Output:
 - Average of array elements: 500000.123456

Problem 7.5: Matrix Transposition using parallel for

- ▶ Description: Transpose a square matrix (e.g., N x N where N = 1000). Create matrix A and store its transpose in matrix B. Implement this in parallel using #pragma omp parallel for. Pay attention to the loop bounds and data access patterns.
- ▶ Requirements:
 - Initialize matrix A.
 - Transpose A into B using parallel loop.
 - Print a small portion of A and B to verify correctness.
- ▶ Sample Input (partial matrix A):
 - A = [[1, 2],
 - [3, 4]]
- ▶ Sample Output (partial matrix B):
 - B = [[1, 3],

- [2, 4]]

Problem 7.6: Combining sections and single

- ▶ Description: Create a parallel region. Inside this region, define three tasks using sections. Additionally, outside the sections but within the same parallel region, use a `#pragma omp single` construct to print a message that should appear only once. Observe the order of execution.
- ▶ Requirements:
 - Use `#pragma omp parallel sections`.
 - Include `#pragma omp section` for at least three distinct tasks.
 - Include a `#pragma omp single` block outside the sections but inside the parallel region.
- ▶ Sample Output (illustrative, order may vary):
 - Thread 0: Starting parallel region.
 - Thread 1: Starting parallel region.
 - Thread 2: Starting parallel region.
 - Task A executed by thread 0
 - Task B executed by thread 1
 - Task C executed by 2
 - Thread 0: Only I execute this 'single' block.
 - Thread 0: After single block.
 - Thread 1: After single block.
 - Thread 2: After single block.

Assessment/Grading (General Guidelines)

- ▶ Code Correctness: Programs compile and run without errors, produce correct results.
- ▶ Understanding of Concepts: Students can explain the purpose of OpenMP directives and functions used.
- ▶ Output Analysis: Students correctly interpret program output and draw conclusions.
- ▶ Performance Analysis: For exercises involving timing, students can compare serial and parallel performance and identify potential bottlenecks.
- ▶ Problem-Solving: Students demonstrate the ability to apply OpenMP concepts to solve parallel problems.
- ▶ Lab Reports: For each lab, students should submit a report including:
 - Code for each exercise (solved and unsolved).
 - Screenshot of the output for each exercise.

- Answers to all questions and discussions posed in the exercises/problems. Observations and insights gained from experiments, especially for performance comparisons.

WEEK #8**Introduction to MPI Environment & Basic Communication**

Objective: To set up the MPI environment and run basic "Hello World" message-passing programs.

Theory/Concepts

- ▶ Introduction to Distributed Memory Parallelism: Why MPI?
- ▶ Processes and Communication: What are MPI processes? How do they communicate?
- ▶ MPI Basic Concepts:
 - Communicators: `MPI_COMM_WORLD`
 - Ranks (Process IDs): `MPI_Comm_rank`
 - Size (Total Processes): `MPI_Comm_size`
 - Initialization and Finalization: `MPI_Init`, `MPI_Finalize`
 - Point-to-Point Communication: `MPI_Send`, `MPI_Recv` (blocking)
- ▶ Compiling and Running MPI Programs: `mpicc`, `mpicxx`, `mpirun`/`mpiexec`.

Lab Setup**1. Download and Install MPI Implementation:**

- For Windows:
 - MPICH: Go to the official MPICH website (<https://www.mpich.org/downloads/>) and download the Windows installer. Follow the installation instructions. Ensure it integrates with your chosen compiler (e.g., MinGW-w64).
 - OpenMPI (less common on Windows, but possible via WSL): If you are using Windows Subsystem for Linux (WSL), you can install OpenMPI within your WSL distribution using `sudo apt-get install openmpi-bin openmpi-common libopenmpi-dev`.
- For Linux/macOS:
 - OpenMPI: `sudo apt-get install openmpi-bin openmpi-common libopenmpi-dev` (Ubuntu/Debian) or `sudo yum install openmpi openmpi-devel` (CentOS/RHEL) or `brew install open-mpi` (macOS).
 - MPICH: Download from <https://www.mpich.org/downloads/> and build from source, or use package managers if available.

2. Verify MPI Installation and Compiler Paths:

- After installation, ensure the bin directory of your MPI installation is in your system's PATH environment variable.
- Open a terminal/command prompt and type `mpiexec -version` or `mpirun -version`. You should see version information.
- Check `mpicc -showme` or `mpicxx -showme` to see what compiler and flags MPI is using.

Alternative MPI Installation Steps (if the above steps do not work for some students):

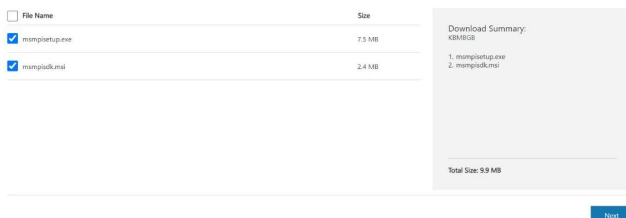
Message Passing Interface (MPI)

Steps:

1. Download Visual Studio from the following link :
 - Visual Studio Community 2022
2. Download useful files for MPI from the following link:
 - Microsoft MPI v10.0
 - Select both files and click on next button
3. After downloading install `msmpisetup.exe` application:
 - Leave all as default and click on next and once the installation is done click on finish
4. Run `msmpisdsk.msi` installer leaving all as default
5. Now run the Visual Studio installer file which was downloaded above.
 - Click Continue to accept the terms and conditions.
 - Once the downloading and installation is done, you are required to sign in using your Microsoft Account.
 - After successful signing in, you will be directed to choose the required workloads. Here we need to select the required workloads.
 - Check the box of required workloads and Click on Install/Modify by leaving the option as Install while downloading.
 - In my case Desktop Development for C++ is enough.
 - It requires nearly 3GB of data to finish whole installation process.
 - Once the installation is done reboot the application.
6. Now after getting restarted, Click on Create a new project and then
 - Select C++ in All Programs drop-down.
 - Now select Console App as shown below and click next
 - Now choose your Project Name and click Create.
 - Call MPI header file: `#include "mpi.h"` as shown below: ○
7. Configuration: After creating project:

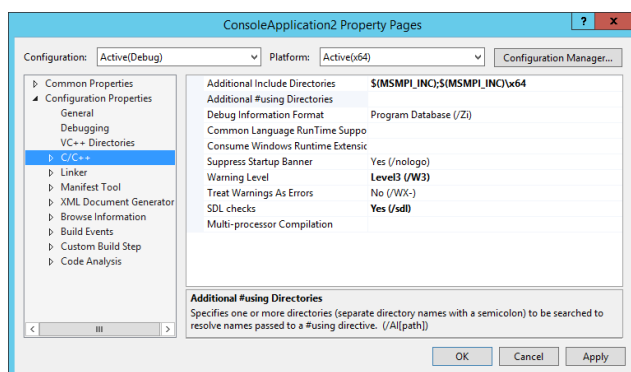
- Head to Project->Properties and go to C/C++ and add \$(MSMPI_INC);\$(MSMPI_INC)\x64 as shown below:
- Now go to Linker->All Options and click scroll to find adding Additional Dependencies and add msmpl.lib and press Apply
- Now in the additional library directories row add \$(MSMPI_LIB64).
- Apply the configurations and then press OK button

MPI Installation and Configuration Screenshots



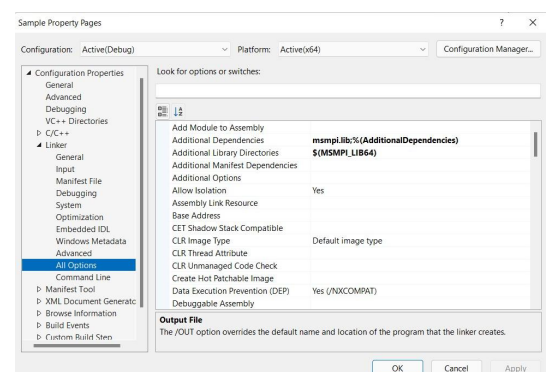
Visual Studio Installer

Getting the Visual Studio Installer ready.



```
#include <iostream>
#include "mpi.h"

int main()
{
    std::cout << "Hello World!\n";
}
```



Original MPI and Visual Studio installation/configuration screenshots retained from the source manual.

Lab Exercises (Solved Examples)

Exercise 8.1: Hello MPI World!

- ▶ Objective: Compile and run a basic MPI program where each process identifies itself.
- ▶ Code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;

    MPI_Init(&argc, &argv); // Initialize MPI environment
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get rank of current process
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes

    printf("Hello from process %d of %d\n", rank, size);

    MPI_Finalize(); // Finalize MPI environment
    return 0;
}
```

- ▶ Tasks:

1. Save the code as hello_mpi.c.
2. Compile using mpicc hello_mpi.c -o hello_mpi.
3. Run using mpiexec -n 4 hello_mpi (for 4 processes).
4. Observe the output. How many "Hello" messages do you see? What is the range of ranks?
5. Try running with a different number of processes (e.g., mpiexec -n 1 hello_mpi, mpiexec -n 8 hello_mpi).
 - ▶ Sample Output (for mpiexec -n 4 hello_mpi):
 - ▶ Hello from process 0 of 4
 - ▶ Hello from process 1 of 4
 - ▶ Hello from process 2 of 4
 - ▶ Hello from process 3 of 4 (Order may vary)

Exercise 8.2: Point-to-Point Communication (Send/Recv)

- ▶ Objective: Implement basic message passing where one process sends data and another receives it.
- ▶ Code:

```

#include <stdio.h>
#include <mpi.h>
#include <string.h> // For strlen and strcpy

int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;
    char message[100];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        fprintf(stderr, "This program requires at least 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    if (rank == 0) { // Sender process
        strcpy(message, "Hello from process 0!");
        MPI_Send(message, strlen(message) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
        printf("Process 0 sent: \"%s\" to process 1\n", message);
    } else if (rank == 1) { // Receiver process
        MPI_Recv(message, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process 1 received: \"%s\" from process 0\n", message);
    }

    MPI_Finalize();
    return 0;
}

```

► Tasks:

1. Compile and run with `mpiexec -n 2 send_recv_mpi`.
2. Observe the communication flow.
3. Modify the code so that process 1 sends a message back to process 0.
4. What happens if you run this with `mpiexec -n 1` or `mpiexec -n 3`? (Processes that don't send/receive will just print their "Hello" and exit, or deadlocks might occur if communication is expected from non-existent processes).
 - Sample Output (for `mpiexec -n 2 send_recv_mpi`):
 - Process 0 sent: "Hello from process 0!" to process 1
 - Process 1 received: "Hello from process 0!" from process 0 (Order of lines may vary, but the send/recv pair will complete.)

Exercise 8.3: Sending and Receiving Different Data Types

- Objective: Practice sending and receiving int and double data types.
- Code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        fprintf(stderr, "This program requires at least 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        int int_val = 123;
        double double_val = 45.67;
        MPI_Send(&int_val, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(&double_val, 1, MPI_DOUBLE, 1, 1, MPI_COMM_WORLD);
        printf("Process 0 sent int %d and double %f to process 1\n", int_val,
double_val);
    } else if (rank == 1) {
        int received_int;
        double received_double;
        MPI_Recv(&received_int, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        MPI_Recv(&received_double, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD, &status);
        printf("Process 1 received int %d and double %f from process 0\n",
received_int,
received_double);
    }

    MPI_Finalize();
    return 0;
}
```

- Tasks:

1. Compile and run with `mpiexec -n 2 data_types_mpi`.
2. Verify that both integer and double values are transferred correctly.
3. Explain the importance of matching MPI datatypes (`MPI_INT`, `MPI_DOUBLE`, `MPI_CHAR`, etc.) between `MPI_Send` and `MPI_Recv`.

Lab Exercises (Unsolved Problems)

Problem 8.1: Round-Robin Message Passing

- Description: Write an MPI program where each process sends a message (e.g., "Hello from X") to the next higher-ranked process (rank i sends to $i+1$). The highest-ranked process sends its message to rank 0. Each process then prints the message it received.
- Requirements:
 - Use `MPI_Send` and `MPI_Recv`.
 - Handle the wrap-around for the last process.
 - Ensure at least 2 processes are used.
- Sample Output (for `mpiexec -n 3`):
- Process 0 received: "Hello from 2"
- Process 1 received: "Hello from 0"
- Process 2 received: "Hello from 1" (Order may vary)

Problem 8.2: Ping-Pong Communication

- Description: Implement a "ping-pong" communication pattern between two processes (rank 0 and rank 1). Rank 0 sends an integer message to rank 1. Rank 1 receives it, increments it, and sends it back to rank 0. This repeats for a fixed number of rounds (e.g., 5 rounds).
- Requirements:
 - Only use 2 processes (`mpiexec -n 2`).
 - Use a loop for multiple rounds.
 - Display the value of the integer at each step.
- Sample Output:
 - Process 0: Sending 0 to Process 1 (round 1)
 - Process 1: Received 0 from Process 0, sending 1 back (round 1)
 - Process 0: Received 1 from Process 1, sending 2 to Process 1 (round 2)
 - Process 1: Received 2 from Process 0, sending 3 back (round 2)
 - ...

Problem 8.3: Master-Worker Setup (Basic)

- ▶ Description: Design a program where process 0 acts as a "master" and all other processes act as "workers". The master sends a unique integer "task ID" to each worker. Each worker receives its task ID, prints a message like "Worker X received task Y", and then sends an acknowledgment message back to the master. The master collects acknowledgments from all workers.
- ▶ Requirements:
 - Master (rank 0) loops from 1 to size-1 to send tasks.
 - Workers (rank $i > 0$) receive their task and send acknowledgment.
 - Master receives acknowledgments in a loop.
- ▶ Sample Output (for `mpiexec -n 3`):
- ▶ Master: Sending task 1 to worker 1
- ▶ Master: Sending task 2 to worker 2
- ▶ Worker 1 received task 1
- ▶ Worker 2 received task 2
- ▶ Worker 1: Acknowledging task 1
- ▶ Worker 2: Acknowledging task 2
- ▶ Master: Received ack from worker 1
- ▶ Master: Received ack from worker 2 (Order of worker messages and master's receive messages may vary)

Problem 8.4: Asynchronous Send/Recv (Conceptual)

- ▶ Description: Research `MPI_Isend` and `MPI_Irecv` (non-blocking communication). Write a simple program (similar to Exercise 7.2) using these non-blocking calls. You'll need `MPI_Wait` or `MPI_Test` to ensure completion. The goal is to understand the concept of non-blocking communication and how it differs from blocking.
- ▶ Requirements:
 - Use `MPI_Isend` and `MPI_Irecv`.
 - Use `MPI_Wait` (or `MPI_Test` in a loop) to wait for communication to complete.
 - Print messages showing when communication is initiated and when it completes.
- ▶ Sample Output:
 - Process 0: Starting non-blocking send.
 - Process 1: Starting non-blocking receive.

- Process 0: Doing some other work while message is in transit...
- Process 1: Doing some other work while message is in transit...
- Process 0: Send completed.
- Process 1: Receive completed. Message: "Hello non-blocking!"

Problem 8.5: Communicator Split

- ▶ Description: Divide `MPI_COMM_WORLD` into two sub-communicators based on whether a process's rank is even or odd. Each process should then print its new rank within its new communicator and its original global rank.
- ▶ Requirements:
 - Use `MPI_Comm_split()`.
 - The color argument should be `rank % 2`.
 - The key argument can be the original rank.
 - Handle cases where a new communicator might be `MPI_COMM_NULL`.
- ▶ Sample Output (for `mpiexec -n 4`):
 - Global Rank 0: New Rank 0 in Even Comm
 - Global Rank 1: New Rank 0 in Odd Comm
 - Global Rank 2: New Rank 1 in Even Comm
 - Global Rank 3: New Rank 1 in Odd Comm

Challenge Problem for Week 8: Distributed Prime Number Check (Partial)

- ▶ Description: Implement a basic distributed prime number checker. Each process P_i will check if numbers in a specific range are prime. Process 0 will distribute a range of numbers to each process. For this week, the distribution can be simple: Process P_i checks numbers from $i * N/\text{size}$ to $(i+1) * N/\text{size} - 1$ (where N is the upper limit). Each process will print the prime numbers it finds within its assigned range.
- ▶ Requirements:
 - Define a `isPrime(int num)` helper function.
 - Each process determines its range based on its rank and size.
 - No communication of results back to a master yet, just printing locally.
- ▶ Sample Output (for `mpiexec -n 4, N=20`):
 - Process 0 checking range [0, 4]
 - Process 0: 2 is prime

- Process 0: 3 is prime
- Process 1 checking range [5, 9]
- Process 1: 5 is prime
- Process 1: 7 is prime
- Process 2 checking range [10, 14]
- Process 2: 11 is prime
- Process 2: 13 is prime
- Process 3 checking range [15, 19]
- Process 3: 17 is prime
- Process 3: 19 is prime
- Discussion Point: What are the limitations of this simple static range distribution for a problem like prime checking? (Hint: Load balancing for primes).

WEEK #9

Collective Communication Operations

Objective: To understand and use fundamental collective communication operations for efficient data sharing among processes.

Theory/Concepts

- ▶ Collective Communication: Operations involving all processes in a communicator.
- ▶ Blocking vs. Non-Blocking Collectives (Brief Mention).
- ▶ Common Collectives:
 - MPI_Bcast: Broadcasts data from one root process to all other processes.
 - MPI_Reduce: Performs a global reduction operation (sum, min, max, etc.) on data from all processes, storing the result on a root process.
 - MPI_Allreduce: Same as MPI_Reduce, but the result is available on all processes.
 - MPI_Gather: Gathers data from all processes onto a single root process.
 - MPI_Scatter: Distributes data from a root process to all other processes.
 - MPI_Barrier: Synchronizes all processes in a communicator.

Lab Exercises (Solved Examples)

Exercise 9.1: MPI_Bcast (Broadcast)

- ▶ Objective: Broadcast a value from rank 0 to all other processes.
- ▶ Code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int data_to_broadcast;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
```

```
        data_to_broadcast = 123;
        printf("Process 0 broadcasting %d\n", data_to_broadcast);
    }
    // All processes call MPI_Bcast. Root is 0.
    MPI_Bcast(&data_to_broadcast, 1, MPI_INT, 0, MPI_COMM_WORLD);

    printf("Process %d received broadcast value: %d\n", rank, data_to_broadcast)
;

    MPI_Finalize();
    return 0;
}
```

► Tasks:

1. Compile and run with `mpiexec -n 4 broadcast_mpi`.
 2. Verify that all processes receive the correct value.
 3. Change the root process (e.g., to rank 1). What happens to the value on rank 0 if it's no longer the root? (It will receive the value from the new root).
- Sample Output (for `mpiexec -n 4 broadcast_mpi`):
- Process 0 broadcasting 123
 - Process 1 received broadcast value: 123
 - Process 2 received broadcast value: 123
 - Process 3 received broadcast value: 123
 - Process 0 received broadcast value: 123 (The `printf` order can vary, but the `Bcast` ensures consistency.)

Exercise 9.2: MPI_Reduce (Summation)

- Objective: Perform a global summation of values, with the result on the root process.
- Code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int local_value;
    int global_sum;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
MPI_Comm_size(MPI_COMM_WORLD, &size);

local_value = rank + 1; // Each process has a unique value
printf("Process %d has local value: %d\n", rank, local_value);

// All processes contribute local_value to a global sum on root (rank 0)
MPI_Reduce(&local_value, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

if (rank == 0) {
    printf("Process 0 (root) calculated global sum: %d\n", global_sum);
}

MPI_Finalize();
return 0;
}
```

► Tasks:

1. Compile and run with `mpiexec -n 4 reduce_sum_mpi`.
2. Verify the correctness of the global sum (e.g., for 4 processes, $1+2+3+4 = 10$).
3. Change the `MPI_SUM` operation to `MPI_MAX` or `MPI_MIN` and observe the result.

► Sample Output (for `mpiexec -n 4 reduce_sum_mpi`):

- Process 0 has local value: 1
- Process 1 has local value: 2
- Process 2 has local value: 3
- Process 3 has local value: 4
- Process 0 (root) calculated global sum: 10

Exercise 9.3: MPI_Allreduce (Average Calculation)

- Objective: Calculate the average of values from all processes, with the result available on all processes.
- Code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    double local_value;
    double global_sum;
```

```
double average;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

local_value = (double)rank * 10.0; // Each process has a unique value
printf("Process %d has local value: %.2f\n", rank, local_value);

// All processes contribute local_value to a global sum on all processes
MPI_Allreduce(&local_value, &global_sum, 1, MPI_DOUBLE, MPI_SUM,
MPI_COMM_WORLD);

average = global_sum / size;
printf("Process %d calculated global average: %.2f\n", rank, average);

MPI_Finalize();
return 0;
}
```

► Tasks:

1. Compile and run with `mpiexec -n 4 allreduce_avg_mpi`.
2. Verify that all processes get the same (correct) average.

► Sample Output (for `mpiexec -n 4 allreduce_avg_mpi`):

- Process 0 has local value: 0.00
- Process 1 has local value: 10.00
- Process 2 has local value: 20.00
- Process 3 has local value: 30.00
- Process 0 calculated global average: 15.00
- Process 1 calculated global average: 15.00
- Process 2 calculated global average: 15.00
- Process 3 calculated global average: 15.00

Lab Exercises (Unsolved Problems)

Problem 9.1: MPI_Gather - Collecting Local Array Parts

- Description: Each process creates a small local array (e.g., 3 integers, `local_data[3]`), where `local_data[i]` is initialized to `rank * 10 + i`. Process 0 should then `MPI_Gather` all these local arrays into a single large array. Process 0 should then print the entire gathered array.

- ▶ Requirements:
 - Use MPI_Gather.
 - Ensure the receive buffer on root (rank 0) is large enough.
 - Print the final array from rank 0.
- ▶ Sample Output (for mpiexec -n 3):
 - Process 0 local: 0 1 2
 - Process 1 local: 10 11 12
 - Process 2 local: 20 21 22
 - Process 0 gathered array: 0 1 2 10 11 12 20 21 22

Problem 9.2: MPI_Scatter - Distributing Array Parts

- ▶ Description: Process 0 has a large array (e.g., 12 integers). It needs to distribute equal portions of this array to all processes using MPI_Scatter. Each process will receive its sub-array and print it.
- ▶ Requirements:
 - The total array size must be divisible by the number of processes.
 - Use MPI_Scatter.
 - Each process prints its received portion.
- ▶ Sample Output (for mpiexec -n 3 with array 0..11):
 - Process 0 received: 0 1 2 3
 - Process 1 received: 4 5 6 7
 - Process 2 received: 8 9 10 11

Problem 9.3: Global Max/Min with MPI_Allreduce

- ▶ Description: Each process generates a random integer. Use MPI_Allreduce to find the global maximum and global minimum of these random numbers, with both results available on all processes.
- ▶ Requirements:
 - Use rand() and srand(time(NULL) + rank) for thread-safe random numbers.
 - Perform two MPI_Allreduce calls: one for MPI_MAX, one for MPI_MIN.
- ▶ Sample Output:
 - Process 0 generated 123. Global Max: 987, Global Min: 123
 - Process 1 generated 456. Global Max: 987, Global Min: 123

- Process 2 generated 987. Global Max: 987, Global Min: 123

Problem 9.4: Synchronizing with MPI_Barrier

- ▶ Description: Each process prints "Before barrier" and then enters an MPI_Barrier. After the barrier, each process prints "After barrier". Introduce an artificial delay (e.g., sleep() or usleep()) in some processes before the barrier to clearly demonstrate its synchronization effect.
- ▶ Requirements:
 - Use MPI_Barrier.
 - Vary sleep duration based on rank (e.g., usleep(rank * 100000)).
 - Observe that "After barrier" messages appear only after all "Before barrier" messages (and delays) are complete.
- ▶ Sample Output (illustrative, with delays):
 - Process 0: Before barrier
 - Process 1: Before barrier
 - Process 2: Before barrier
 - Process 3: Before barrier
 - (short pause due to delays)
 - Process 0: After barrier
 - Process 1: After barrier
 - Process 2: After barrier
 - Process 3: After barrier

Problem 9.5: Parallel Pi Calculation (Monte Carlo)

- ▶ Description: Estimate the value of Pi using the Monte Carlo method. Each process generates a certain number of random points within a unit square. It counts how many of these points fall within a unit circle. Then, use MPI_Reduce to sum up the counts from all processes, and the root process calculates the final Pi estimate.
- ▶ Requirements:
 - Generate random x, y coordinates between 0 and 1.
 - Check if $x^2 + y^2 \leq 1$.
 - Each process calculates a local_hits.
 - Use MPI_Reduce to get global_hits.
 - Formula for Pi: $4.0 * \text{global_hits} / \text{total_points}$.

- Use MPI_Wtime() to measure execution time.
- ▶ Sample Output:
 - Process 0 local hits: 12345
 - Process 1 local hits: 12350
 - Process 2 local hits: 12340
 - Process 3 local hits: 12360
 - Total points: 400000
 - Estimated Pi: 3.14150
 - Execution Time: 0.012345 seconds

Challenge Problem for Week 9: Parallel Matrix-Vector Multiplication

- ▶ Description: Implement a parallel matrix-vector multiplication $y = Ax$ using MPI. The matrix A (size $N \times N$) and vector x (size $N \times 1$) are given. Distribute the rows of matrix A among the processes. Each process will then compute its portion of the result vector y. Finally, gather all parts of y back to process 0.
- ▶ Requirements:
 - Assume N is divisible by size.
 - Process 0 initializes the full matrix A and vector x.
 - Use MPI_Scatter to distribute rows of A and the full vector x (if needed, or broadcast x).
 - Each process computes its part of y.
 - Use MPI_Gather to collect the partial results of y on process 0.
 - Print a small portion of the resulting vector y from process 0 to verify.
- ▶ Sample Input (Illustrative, for $N=4$, size=2):
 - $A = \begin{bmatrix} 1,2,3,4, \\ 5,6,7,8, \\ 9,0,1,2, \\ 3,4,5,6 \end{bmatrix}$
 - $x = [1,1,1,1]$
- ▶ Sample Output (Illustrative):
 - ▶ Process 0: Scatter A part: $\begin{bmatrix} 1,2,3,4, \\ 5,6,7,8 \end{bmatrix}$
 - ▶ Process 1: Scatter A part: $\begin{bmatrix} 9,0,1,2, \\ 3,4,5,6 \end{bmatrix}$
 - ▶ Process 0: Computed $y_part = [10, 26]$
 - ▶ Process 1: Computed $y_part = [12, 18]$

- ▶ Process 0: Final $y = [10, 26, 12, 18]$ (Expected $y=[10,26,12,18]$).

WEEK #10**Parallel Numerical Methods & Advanced Sorting (OpenMP Focus)**

Objective: To implement and analyze parallel numerical methods for solving linear systems and to explore advanced parallel sorting algorithms using OpenMP for shared-memory parallelism.

Theory/Concepts

- ▶ Iterative Solvers for Linear Systems: Introduction to methods like Jacobi and Gauss-Seidel.
- ▶ Data Distribution Strategies: Implicit data distribution for shared-memory (all threads access common arrays).
- ▶ Synchronization in Iterative Methods: Importance of barriers (`#pragma omp barrier`), critical sections (`#pragma omp critical`), and careful handling of data dependencies to ensure correctness (e.g., for Gauss-Seidel).
- ▶ Convergence Criteria: Methods to determine when an iterative process has reached a sufficient solution (e.g., residual norm, $\|x(k+1)-x(k)\|_2$).
- ▶ Parallel Sorting Algorithms: Overview of different strategies for parallel sorting (e.g., comparison sorts, network sorts).
- ▶ Bitonic Sort: Principles of bitonic sequences and merging networks. Its suitability for parallelization due to predictable communication patterns and regular structure.
- ▶ OpenMP Directives Revisited: In-depth use of `#pragma omp parallel`, `#pragma omp parallel for`, `#pragma omp single`, `omp_get_wtime()`, and discussion of schedule clauses (static, dynamic, guided).
- ▶ Performance Metrics: Detailed analysis of Speedup ($Sp=Ts/Tp$), Efficiency ($Ep = Sp/p$), and understanding Amdahl's Law in practical scenarios.

Lab Exercises (Solved Example)**Problem 10.1 Parallel Bitonic Sort (OpenMP)**

- ▶ Description: Implement and analyze the Bitonic Sort algorithm in parallel using OpenMP. Bitonic Sort is a comparison-based sorting algorithm that works well on parallel computers due to its predictable pattern of comparisons.
- ▶ Requirements:
- ▶ Initialize OpenMP: Set up the OpenMP environment and create an array of random integers whose size is a power of 2 (e.g., $N=2^k$, for k between 10 and 20). Use `malloc` for dynamic array allocation.
- ▶ Bitonic Sequence Concept: Understand that a bitonic sequence is a sequence that first monotonically increases then monotonically decreases, or vice-versa.

- ▶ **Bitonic Compare Function:** Implement a void `compare(int *arr, int i, int j, int dir)` function that compares and potentially swaps `arr[i]` and `arr[j]` based on the `dir` (direction: 1 for ascending, 0 for descending).
- ▶ **Bitonic Merge Function:** Implement a void `bitonic_merge(int *arr, int low, int cnt, int dir)` function. This function takes a bitonic sequence of length `cnt` starting at `low` and sorts it in the given `dir`. This involves comparing elements `cnt/2` apart, then recursively calling `bitonic_merge` on the two halves. The comparison loop within `bitonic_merge` is a key parallelization point.
- ▶ **Bitonic Sort Algorithm:** Implement the main void `bitonic_sort_recursive(int *arr, int low, int cnt, int dir)` function. This recursively splits the array into two halves, sorts one in ascending and the other in descending order to form a bitonic sequence, and then merges them.
- ▶ **Parallelization with OpenMP:** Use `#pragma omp parallel` combined with `#pragma omp single` to initiate the sort, and `#pragma omp for` within the `bitonic_merge` function for parallel comparisons.
- ▶ **Compare Results:** Validate the sorted array. Compare the performance of the parallel Bitonic Sort with a standard sequential sort (`qsort`).
- ▶ **Optimize Performance:** Experiment with different OpenMP scheduling clauses (`schedule(static)`, `schedule(dynamic)`, `schedule(guided)`) for your `#pragma omp for` directives. Observe the impact of the number of threads on speedup and efficiency.
- ▶ **Solved Code Example:** `bitonic_sort_solution.c`

```
#include <stdio.h>
#include <stdlib.h> // For malloc, free, srand, rand, qsort
#include <time.h> // For time
#include <math.h> // For log2
#include <omp.h> // For OpenMP

// Helper function to swap two integers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to compare and swap elements based on direction
// dir = 1 for ascending, 0 for descending
void compareAndSwap(int *arr, int i, int j, int dir) {
    if ((arr[i] > arr[j] && dir == 1) || (arr[i] < arr[j] && dir == 0)) {
        swap(&arr[i], &arr[j]);
    }
}

// Function to merge a bitonic sequence
```

```

// 'arr' is the array, 'low' is the starting index, 'cnt' is the count of elements
// 'dir' is the sorting direction (1 for ascending, 0 for descending)
void bitonic_merge(int *arr, int low, int cnt, int dir) {
    if (cnt > 1) {
        int k = cnt / 2; // Half length of the current sequence

        // Parallelize the comparison phase of the merge
        // This loop iterates through the first half and compares with the second half
        // Each comparison is independent, making it suitable for parallelization.
        #pragma omp for // This directive must be inside a parallel region
        for (int i = low; i < low + k; ++i) {
            compareAndSwap(arr, i, i + k, dir);
        }

        // Recursive calls for the two halves. These are still sequential in this model
        // but the work within them (sub-merges) can be parallelized again.
        bitonic_merge(arr, low, k, dir);
        bitonic_merge(arr, low + k, k, dir);
    }
}

// Function to recursively perform bitonic sort
// 'arr' is the array, 'low' is the starting index, 'cnt' is the count of elements
// 'dir' is the sorting direction for the final merge (1 for ascending, 0 for
// descending)
void bitonic_sort_recursive(int *arr, int low, int cnt, int dir) {
    if (cnt > 1) {
        int k = cnt / 2;
        // Step 1: Recursively sort first half in ascending, second half in descending
        // This forms a bitonic sequence
        bitonic_sort_recursive(arr, low, k, 1); // Sort first half ascending
        bitonic_sort_recursive(arr, low + k, k, 0); // Sort second half descending

        // Step 2: Merge the two bitonic halves into a single sorted bitonic sequence
        bitonic_merge(arr, low, cnt, dir);
    }
}

// Wrapper function to initiate the parallel bitonic sort
void parallel_bitonic_sort(int *arr, int N, int num_threads) {
    // Set the number of threads for the upcoming parallel region
    omp_set_num_threads(num_threads);

    // Create a parallel region. Only one thread (the master) will execute the single
    // block.
    // The #pragma omp for inside bitonic_merge will then be executed by the team of

```

```

threads.
#pragma omp parallel
{
    #pragma omp single // Ensure only one thread initiates the recursive sort
    {
        bitonic_sort_recursive(arr, 0, N, 1); // Start sorting the entire array in
        ascending order
    }
}

// Comparison function for qsort
int compare_qsort(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Function to check if array is sorted
int is_sorted(const int *arr, int N) {
    for (int i = 0; i < N - 1; ++i) {
        if (arr[i] > arr[i + 1]) {
            return 0; // Not sorted
        }
    }
    return 1; // Sorted
}

int main() {
    // Array sizes to test (must be powers of 2)
    // For larger N, expect more significant speedup
    int N_vals[] = {1024, 4096, 16384, 65536, 262144, 1048576};
    int num_N_vals = sizeof(N_vals) / sizeof(N_vals[0]);
    int num_threads_vals[] = {1, 2, 4, 8}; // Number of threads to test
    int num_num_threads_vals = sizeof(num_threads_vals) / sizeof(num_threads_vals[0]);

    printf("Parallel Bitonic Sort Analysis\n");
    printf("-----\n");

    for (int i_N = 0; i_N < num_N_vals; ++i_N) {
        int N = N_vals[i_N];
        printf("\n--- Sorting N = %d elements ---\n", N);

        int *arr = (int *)malloc(N * sizeof(int));
        int *arr_seq = (int *)malloc(N * sizeof(int)); // For sequential comparison

        if (arr == NULL || arr_seq == NULL) {
            fprintf(stderr, "Memory allocation failed for N=%d\n", N);

```

```

        return 1;
    }

    // Initialize array with random values
    srand(time(NULL)); // Seed random number generator
    for (int i = 0; i < N; ++i) {
        arr[i] = rand() % 10000; // Values between 0 and 9999
    }

    // --- Sequential Sort (qsort) for baseline ---
    // Copy original array to arr_seq for sequential sort
    for(int i = 0; i < N; ++i) arr_seq[i] = arr[i];

    double start_time_seq_qsort = omp_get_wtime();
    qsort(arr_seq, N, sizeof(int), compare_qsort);
    double end_time_seq_qsort = omp_get_wtime();
    double seq_qsort_time = end_time_seq_qsort - start_time_seq_qsort;
    printf("Sequential qsort time: %lf seconds\n", seq_qsort_time);
    if (!is_sorted(arr_seq, N)) {
        printf("Error: qsort failed to sort array!\n");
    }

    // --- Parallel Bitonic Sort for various thread counts ---
    for (int i_threads = 0; i_threads < num_num_threads_vals; ++i_threads) {
        int num_threads = num_threads_vals[i_threads];
        if (num_threads > omp_get_max_threads() && num_threads > 1) {
            // Avoid testing more threads than available or reasonable
            printf("Skipping Threads=%d (too many for system or N).\n", num_threads);
            continue;
        }

        int *arr_par = (int *)malloc(N * sizeof(int)); // New array for parallel run
        if (arr_par == NULL) {
            fprintf(stderr, "Memory allocation failed for arr_par, N=%d\n", N);
            free(arr); free(arr_seq);
            return 1;
        }
        // Reset array for each parallel run
        for(int i = 0; i < N; ++i) arr_par[i] = arr[i];

        double start_time_par = omp_get_wtime();
        parallel_bitonic_sort(arr_par, N, num_threads);
        double end_time_par = omp_get_wtime();

        double par_time = end_time_par - start_time_par;
    }

```

```

// Calculate speedup relative to qsort (or your own single-threaded bitonic sort)
double speedup = (par_time > 0) ? seq_qsort_time / par_time : 0.0;
double efficiency = (speedup > 0) ? speedup / num_threads : 0.0;

printf("Parallel Bitonic Sort (Threads=%d) Time: %lf seconds", num_threads,
par_time);
printf(", Speedup (vs qsort): %lf, Efficiency: %lf\n", speedup, efficiency);

// Validate sorted array
if (is_sorted(arr_par, N)) {
    printf("Array sorted correctly: Yes\n");
} else {
    printf("Array sorted correctly: No (Error in parallel sort!)\n");
    }
    free(arr_par); // Free memory for this parallel run
    }
    free(arr); // Free original array
    free(arr_seq); // Free sequential array
}

return 0;
}

```

Expected Output Characteristics:

- ▶ Similar to the C++ version, for small N, the overhead of OpenMP might make the parallel version slower than qsort and even slower than its own 1-thread parallel run. Speedup and efficiency will be low or less than 1.
- ▶ As N increases (e.g., 65536, 1048576), the parallel overhead becomes relatively smaller, and you should start seeing speedup greater than 1, and efficiency will improve (though rarely reaching 1.0 due to overhead and sequential portions).
- ▶ qsort is a highly optimized library function. Your custom Bitonic Sort, even parallel, might not beat qsort in absolute time for all N due to constant factors, but the key is the speedup of your parallel version over its own single-threaded execution (i.e., when num_threads=1 for your parallel_bitonic_sort function).

Lab Exercises (Unsolved Problems)

1. Implement a parallel version of the Gauss-Seidel method for solving a system of linear equations. This involves distributing the matrix and vector among processes, performing iterative updates in parallel, and ensuring synchronization among processes.
2. Compare the performance of different methods for solving linear equations.

WEEK #11**Advanced Parallel Sorting (OpenMP & MPI)**

Objective: To implement and analyze additional parallel sorting algorithms, specifically Quick Sort using OpenMP and Merge Sort using both OpenMP and MPI, focusing on parallelization strategies and load balancing.

Theory/Concepts

- ▶ Quick Sort Parallelization: Strategies for parallelizing the recursive calls (e.g., using OpenMP tasks or sections) and challenges related to partitioning.
- ▶ Merge Sort Parallelization: Parallelizing the recursive divide step and the merge step. Distributed-memory merge sort (MPI) will highlight data communication and aggregation.
- ▶ Load Balancing in Parallel Algorithms: Importance of distributing work evenly across threads/processes to minimize idle time. Static vs. Dynamic load balancing.
- ▶ MPI Communication Primitives:
 - Point-to-Point: MPI_Send, MPI_Recv for transferring data between two specific processes.
 - Collective Communications: MPI_Scatter (distribute data from one to all), MPI_Gather (collect data from all to one), MPI_Bcast (broadcast from one to all), MPI_Reduce (perform reduction operation across all processes).
- ▶ Hybrid Parallelism (Optional/Advanced): A brief introduction to combining MPI and OpenMP.

Lab Exercises (Partially Solved Example)**Problem 11.1: Parallel Quick Sort (OpenMP)**

- ▶ Description: Implement the Quick Sort algorithm in parallel using OpenMP to sort an array of integers. Explore how parallelism affects performance and how to manage recursive parallelism.
 - ▶ Requirements:
1. Initialize OpenMP: Set up the OpenMP environment and create an array of random integers of a significant size.
 2. Implement Partition Function: The standard partition function (Lomuto or Hoare) should be implemented sequentially. Its performance is often a bottleneck for small sub- arrays.

3. **Implement Parallel Quick Sort:** Implement the recursive Quick Sort algorithm using OpenMP tasks.
 - **Strategy:** The key is to parallelize the recursive calls. After the partition step, the two resulting sub-arrays can be sorted independently. Use `#pragma omp task` for each recursive call.
 - **Task Cutoff:** To prevent excessive task creation overhead for small subarrays, introduce a `THRESHOLD_SIZE`. If a subarray's size falls below this, sort it sequentially using `std::sort` or a sequential Quick Sort.
 - **Initial Task Creation:** Ensure that only one thread (the master thread) initially creates the first task for the entire array. Subsequent tasks are created by any thread that executes a task.
4. **Compare Results:** Validate the sorted array. Compare the performance of the parallel Quick Sort with a sequential implementation (`std::sort` or your own sequential Quick Sort).
5. **Optimize Performance:** Experiment with different `THRESHOLD_SIZE` values. Analyze the impact of this threshold on performance.

Partial Solved Code Example (Key Snippets): This provides the core structure for parallelizing Quick Sort with OpenMP tasks. You'll need to complete the partition function and the overall main function with timing, array generation, and validation.

```
#include <stdio.h>
#include <stdlib.h> // For malloc, free, srand, rand, qsort
#include <time.h> // For time
#include <omp.h> // For OpenMP

// Define a threshold for switching to sequential sort
// This is crucial for performance with OpenMP tasks
const int THRESHOLD_SIZE = 1024; // Adjust based on experimentation

// Helper function to swap two integers
void swap_int(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Comparison function for qsort
int compare_qsort_int(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Standard sequential partition function (Lomuto partition scheme)
int partition_arr(int *arr, int low, int high) {
    int pivot = arr[high]; // Choose last element as pivot
```

```

    int i = (low - 1); // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++; // Increment index of smaller element
            swap_int(&arr[i], &arr[j]);
        }
    }
    swap_int(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// Parallel Quick Sort using OpenMP tasks
void parallel_quick_sort_recursive(int *arr, int low, int high) {
    if (low < high) {
        // If subarray is small, sort sequentially to avoid task overhead
        if (high - low + 1 <= THRESHOLD_SIZE) {
            qsort(arr + low, high - low + 1, sizeof(int), compare_qsort_int);
            return;
        }

        // Partition the array
        int pi = partition_arr(arr, low, high);

        // Create tasks for recursive calls
        // #pragma omp task will create a new task for each recursive call.
        // These tasks can then be executed by available threads in the team.
        #pragma omp task
        parallel_quick_sort_recursive(arr, low, pi - 1); // Sort left subarray

        #pragma omp task
        parallel_quick_sort_recursive(arr, pi + 1, high); // Sort right subarray
    }
}

// Wrapper function to initiate the parallel Quick Sort
void initiate_parallel_quick_sort(int *arr, int N, int num_threads) {
    omp_set_num_threads(num_threads);

    // This creates the parallel region and ensures only one thread
    // initially creates the first task for the entire array.
    #pragma omp parallel
    {
        #pragma omp single nowait // 'nowait' allows other threads to pick up tasks immediately
    }
}

```

```
        {
            parallel_quick_sort_recursive(arr, 0, N - 1);
        }
    }
    // All threads in the team will implicitly wait here for all tasks generated
    // by this parallel region to complete, or use #pragma omp taskwait
    // if a more explicit synchronization is needed within a single region.
    #pragma omp taskwait // Ensure all tasks generated are completed before exiting
    parallel region
}

// Function to check if array is sorted (same as Week 9)
int is_sorted(const int *arr, int N) {
    for (int i = 0; i < N - 1; ++i) {
        if (arr[i] > arr[i + 1]) {
            return 0; // Not sorted
        }
    }
    return 1; // Sorted
}

// --- main function structure (to be completed by student) ---
int main() {
    int N_vals[] = {100000, 500000, 1000000, 5000000}; // Large array sizes
    int num_N_vals = sizeof(N_vals) / sizeof(N_vals[0]);
    int num_threads_vals[] = {1, 2, 4, 8}; // Number of threads to test
    int num_num_threads_vals = sizeof(num_threads_vals) / sizeof(num_threads_vals[0]);

    printf("Parallel Quick Sort Analysis\n");
    printf("-----\n");

    for (int i_N = 0; i_N < num_N_vals; ++i_N) {
        int N = N_vals[i_N];
        printf("\n--- Sorting N = %d elements ---\n", N);

        int *arr_original = (int *)malloc(N * sizeof(int));
        if (arr_original == NULL) {
            fprintf(stderr, "Memory allocation failed for arr_original, N=%d\n", N);
            return 1;
        }

        // Initialize array with random values
        srand(time(NULL));
        for (int i = 0; i < N; ++i) {
            arr_original[i] = rand() % 1000000;
        }
    }
}
```

```
// --- Sequential Sort (qsort) for baseline ---
int *arr_seq = (int *)malloc(N * sizeof(int));
if (arr_seq == NULL) {
    fprintf(stderr, "Memory allocation failed for arr_seq, N=%d\n", N);
    free(arr_original);
    return 1;
}
for(int i = 0; i < N; ++i) arr_seq[i] = arr_original[i];

double start_time_seq_qsort = omp_get_wtime();
qsort(arr_seq, N, sizeof(int), compare_qsort_int);
double end_time_seq_qsort = omp_get_wtime();
double seq_qsort_time = end_time_seq_qsort - start_time_seq_qsort;
printf("Sequential qsort time: %lf seconds\n", seq_qsort_time);
if (!is_sorted(arr_seq, N)) {
    printf("Error: qsort failed to sort array!\n");
}
free(arr_seq); // Free sequential array

// --- Parallel Quick Sort for various thread counts ---
for (int i_threads = 0; i_threads < num_num_threads_vals; ++i_threads) {
    int num_threads = num_threads_vals[i_threads];

    int *arr_par = (int *)malloc(N * sizeof(int));
    if (arr_par == NULL) {
        fprintf(stderr, "Memory allocation failed for arr_par, N=%d\n", N);
        free(arr_original);
        return 1;
    }
    // Copy original array for parallel run
    for(int i = 0; i < N; ++i) arr_par[i] = arr_original[i];

    double start_time_par = omp_get_wtime();
    initiate_parallel_quick_sort(arr_par, N, num_threads);
    double end_time_par = omp_get_wtime();

    double par_time = end_time_par - start_time_par;
    double speedup = (par_time > 0) ? seq_qsort_time / par_time : 0.0;
    double efficiency = (speedup > 0) ? speedup / num_threads : 0.0;

    printf("Parallel Quick Sort (Threads=%d) Time: %lf seconds", num_threads,
par_time);
    printf(", Speedup (vs qsort): %lf, Efficiency: %lf\n", speedup, efficiency)
;
}
```

```
        if (is_sorted(arr_par, N)) {
            printf("Array sorted correctly: Yes\n");
        } else {
            printf("Array sorted correctly: No (Error in parallel sort!)\n");
        }
        free(arr_par); // Free parallel array
    }
    free(arr_original); // Free original array
}

return 0;
}
```

Lab Exercises (Unsolved Problems)

Problem 11.1: Parallel Merge Sort (OpenMP and MPI with Load Balancing)

- Description: Implement the Merge Sort algorithm using OpenMP (shared-memory) and MPI (distributed-memory). Analyze the impact of load balancing strategies, particularly for the MPI version.

- Requirements:

1. OpenMP Implementation:

- Recursive Parallelization: Similar to Quick Sort, parallelize the recursive calls of Merge Sort using `#pragma omp task` or `#pragma omp sections`.
- Parallel Merge (Optional/Advanced): For larger sub-arrays, consider a parallel merge strategy, but for simplicity, sequential merge after parallel sorts is often sufficient for good speedup.
- Measure performance for different numbers of OpenMP threads.

2. MPI Implementation (Distributed Memory):

- Initialization: Initialize MPI (`MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size`).
- Data Distribution (`MPI_Scatter`): The root process (rank 0) generates the full unsorted array and then scatters equal-sized chunks to all processes using `MPI_Scatter`. Ensure proper handling of array sizes that are not perfectly divisible by the number of processes.
- Local Sort: Each MPI process sorts its local chunk sequentially (e.g., using `std::sort`).
- Parallel Merge (Tree-based / Logarithmic Stages):
- Implement a series of merge steps. In $\log_2(P)$ stages (where P is the number of processes):
- Pairs of processes (e.g., rank and rank + 2^s) communicate.

- One process sends its sorted local chunk to its partner.
- The receiving process merges its own sorted chunk with the received chunk.
- After each stage, half the processes become "inactive" or simply hold larger sorted chunks, doubling the sorted segment size until only one process (rank 0) holds the fully sorted array.
- Load Balancing:
 - Initial Data Distribution: Focus on distributing the initial data as evenly as possible using `MPI_Scatterv` if necessary for uneven chunk sizes.
 - Merge Strategy: The tree-based merge inherently provides some load balancing by distributing merge operations.
 - Performance Measurement: Use `MPI_Wtime()` to measure execution time for varying numbers of MPI processes (`mpirun -np P ./your_mpi_program`).
 - Final Collection (`MPI_Gather`): The root process gathers all sorted chunks (or receives the final sorted array) using `MPI_Gather` or a final `MPI_Send/MPI_Recv`.

3. Comparison and Analysis:

- Compare the performance (speedup, efficiency) of the sequential, OpenMP parallel, and MPI parallel implementations.
- Analyze the impact of communication overhead on the MPI version.
- Discuss the challenges and advantages of each parallel paradigm for Merge Sort.
- Explain how the MPI communication patterns contribute to or hinder load balancing.

General Instructions for Students:

- ▶ Code Structure: Organize your code into well-named functions. Use meaningful variable names.
- ▶ Comments: Add clear and concise comments to your code, explaining the logic, parallelization directives, and any critical synchronization points.
- ▶ Output and Analysis: Ensure your programs print execution times, speedup, and efficiency. For problems involving scalability, generate graphs (you can paste data into Excel/Python for plotting).
- ▶ Experimentation: Crucially, experiment with different array sizes, numbers of threads/processes, and relevant parameters (e.g., `THRESHOLD_SIZE`, OpenMP schedule clauses). Document how these changes affect performance.
- ▶ Validation: Always include code to verify the correctness of your sorted arrays or the accuracy of your numerical solutions.
- ▶ Self-Correction: Learn to debug parallel code. Understand race conditions, deadlocks, and common OpenMP/MPI errors.
- ▶ Report: For each problem, prepare a brief report summarizing your approach, method-

ology, observed results (with tables/graphs), and a discussion of your findings (speedup, efficiency, bottlenecks, comparison of methods).

WEEK #12**Cloud Security Fundamentals & Basic Web Services**

Objective: To understand and demonstrate core cloud security concepts, particularly data encryption and user authentication, and to implement a basic web service in a cloud environment.

Theory/Concepts

- ▶ Cloud Security Shared Responsibility Model: Understanding the division of security responsibilities between cloud provider and customer.
- ▶ Data Encryption in Cloud: Concepts of encryption at rest (storage) and encryption in transit (network). Key management services (KMS).
- ▶ Cloud Identity and Access Management (IAM): Principles of user authentication, authorization, roles, policies, and multi-factor authentication (MFA).
- ▶ Cloud Security Tools: Overview of various security services offered by cloud providers (e.g., WAF, DDoS protection, security groups, network ACLs, compliance tools).
- ▶ Web Services Basics: Understanding HTTP/HTTPS, RESTful APIs, basic client-server architecture.
- ▶ Cloud Deployment Models for Web Services: IaaS vs. PaaS for web applications (e.g., VMs vs. App Services/Elastic Beanstalk/Cloud Functions).

Lab Setup

- ▶ Access to a cloud provider account (AWS/Azure/GCP).
- ▶ Chosen programming language (e.g., Python, Node.js) and relevant SDKs/libraries installed.

Lab Exercises (Unsolved Problems)**Problem 12.1: Demonstrate Data Encryption in Cloud Storage**

- ▶ Description: This exercise aims to demonstrate how data stored in a cloud environment can be encrypted to enhance security. You will use a cloud provider's native encryption features.
 - ▶ Requirements:
1. Choose a Cloud Storage Service: Select a block storage (e.g., EBS volume on AWS, Azure Disk) or object storage (e.g., S3 bucket on AWS, Azure Blob Storage, GCP Cloud Storage).

2. Enable Encryption at Rest:

- For Object Storage: Create a new storage bucket/container and enable server-side encryption (e.g., S3-managed keys (SSE-S3), KMS-managed keys (SSE-KMS), or customer-provided keys (SSE-C)). Upload a sample file (e.g., a text document or image) to this encrypted storage.
- For Block Storage: Create an encrypted disk volume. Attach it to a virtual machine. Write some data to the volume.

- 3. Demonstrate Encryption in Transit (Optional but Recommended):** Access the uploaded object/data via HTTPS to show data encryption during transfer over the network.
- 4. Verification:** Describe the steps taken and provide screenshots or CLI output confirming that encryption is enabled for the stored data.

► Discussion Points:

- 1.** Explain the difference between server-side and client-side encryption.
- 2.** Discuss the benefits and trade-offs of using cloud-managed encryption keys versus customer-managed keys (CMK) or customer-provided keys (CPK).
- 3.** How does encryption contribute to data privacy and compliance in the cloud?

Problem 12.2: Compare Different Cloud Security Tools

- Description: Research and compare at least three different cloud security tools or services offered by major cloud providers (e.g., AWS, Azure, GCP). Focus on their purpose, features, and target security concerns.

► Requirements:

- 1. Identify Tools:** Select three distinct security tools/services (e.g., a Web Application Firewall (WAF), a Security Information and Event Management (SIEM) service, a Vulnerability Management service, a DDoS protection service, or an Identity and Access Management (IAM) service).
- 2. Comparative Analysis:** For each tool, provide:
 - A brief description of its primary function.
 - Key features and capabilities.
 - The specific security problems it aims to solve.
 - Its typical integration points within a cloud architecture.
 - Comparison of how different providers might implement similar tools (e.g., AWS WAF vs. Azure Application Gateway WAF).
- 3. Scenario Application (Optional):** Discuss a hypothetical cloud deployment scenario and explain which of these tools would be most relevant and why.

► Discussion Points:

1. Which types of threats are most effectively mitigated by each tool?
2. How do these tools contribute to a multi-layered security strategy in the cloud?
3. Discuss the challenges of integrating and managing multiple security tools in a complex cloud environment.

Problem 12.3: Implement User Authentication in a Cloud Environment

► Description: Write a program to implement basic user authentication, leveraging a cloud identity service or a simple backend deployed on a cloud platform.

► Requirements:

1. Choose a Cloud Authentication Method:
 - Option A (Recommended): Use a cloud-native identity service (e.g., AWS Cognito, Azure Active Directory B2C, Google Identity Platform) to manage users.
 - Option B: Implement a basic backend (e.g., a Python Flask/Django app, Node.js Express app) on a cloud VM or serverless function that handles user registration and login (storing credentials securely, e.g., hashed passwords in a cloud database).
2. Program Functionality:
 - User Registration: Allow new users to sign up with a username and password.
 - User Login: Authenticate existing users.
 - Simple Protected Resource: After successful authentication, display a simple "Welcome, [username]!" message or access a trivial protected resource.
3. Deployment: Deploy your program to the cloud (e.g., a VM, a serverless function like AWS Lambda/Azure Functions/GCP Cloud Functions, or a container on a managed service like AWS Fargate).
4. Code & Demonstration: Provide the source code and demonstrate the registration and login flow.
 - Discussion Points:
 1. What are the advantages of using a cloud-native identity service over implementing your own authentication system?
 2. Discuss different authentication methods (e.g., password-based, token-based, OAuth, OpenID Connect) and their suitability for cloud environments.
 3. How would you enhance the security of this basic authentication system (e.g., MFA, password policies, rate limiting)?

Problem 12.4: Implement Basic Web Service on Cloud Environment

► Description: Implement and deploy a basic web service (API) to a cloud environment.

The web service should expose at least one RESTful endpoint.

► Requirements:

1. Choose a Cloud Service Type:
 - Option A (PaaS/Serverless): Use a Platform as a Service (e.g., AWS Elastic Beanstalk, Azure App Service, Google App Engine) or Serverless Function (e.g., AWS Lambda + API Gateway, Azure Functions, Google Cloud Functions).
 - Option B (IaaS): Deploy on a Virtual Machine (e.g., AWS EC2, Azure VM, GCP Compute Engine) with a web server (e.g., Nginx, Apache) and your application framework (e.g., Flask, Node.js Express).
 2. Web Service Functionality:
 - Implement a simple RESTful GET endpoint (e.g., /hello which returns "Hello, World!" or /greet?name=User which returns "Hello, User!").
 - (Optional) Implement a POST endpoint (e.g., /items to add an item to a list stored in memory or a simple file).
 3. Deployment: Deploy your web service to the chosen cloud environment and make it publicly accessible via HTTP/HTTPS.
 4. Testing: Test the web service using a web browser, curl, or Postman. Provide screenshots of the successful API calls.
 5. Code: Provide the source code for your web service.
- Discussion Points:
1. Compare the benefits and drawbacks of deploying a web service using PaaS/Serverless versus IaaS.
 2. How would you scale this web service to handle increased traffic?
 3. Discuss common security considerations for web services deployed in the cloud (e.g., API security, input validation).

WEEK #13**Cloud Data Analytics & Advanced Security**

Objective: To demonstrate the application of cloud computing in data analytics and to deepen the understanding of cloud security concepts through further exploration of encryption and security tools.

Theory/Concepts

- ▶ Cloud Data Analytics Pipeline: Overview of typical steps (data ingestion, storage, processing, analysis, visualization).
- ▶ Big Data Services in Cloud: Introduction to managed services for data warehousing (e.g., Redshift, Synapse Analytics, BigQuery), big data processing (e.g., EMR, HDInsight, Dataproc), stream processing (e.g., Kinesis, Event Hubs, Pub/Sub), and analytics tools.
- ▶ Data Encryption Revisited: Deeper dive into client-side encryption, key hierarchies, and key rotation.
- ▶ Advanced Cloud Security Tools: Further exploration of specific security services (e.g., identity federation, security dashboards, compliance auditing tools).
- ▶ User Authentication Revisited: Exploring more complex scenarios like role-based access control (RBAC), fine-grained authorization, and integration with enterprise directories.

Lab Setup

- ▶ Access to a cloud provider account (AWS/Azure/GCP).
- ▶ Chosen programming language (e.g., Python) and relevant SDKs/libraries installed.

Lab Exercises (Unsolved Problems)**Problem 13.1: Demonstrate Data Encryption in Cloud Storage (Advanced)**

- ▶ Description: Building on Week 12, this exercise focuses on demonstrating more advanced data encryption techniques or using a Key Management Service (KMS).
 - ▶ Requirements:
1. Use a Key Management Service (KMS): Create a customer-managed key (CMK) within your cloud provider's KMS.
 2. Encrypt Data with CMK: Use this CMK to encrypt an object in your cloud storage service (e.g., S3, Blob Storage, Cloud Storage). This might involve specifying the KMS key when uploading the object.

3. Demonstrate Key Rotation (Conceptual/Config): Explain or configure an automatic key rotation policy for your CMK in KMS.
4. Access with Permissions: Show how access to the encrypted data is governed by IAM permissions related to both the storage and the KMS key.
 - ▶ Discussion Points:
 1. What are the advantages of using a KMS for managing encryption keys?
 2. Discuss the importance of key rotation and how it enhances security.
 3. How does the interplay between IAM and KMS ensure granular control over encrypted data?

Problem 13.2: Demonstrate the Use of Cloud Computing in Data Analytics

- ▶ Description: Utilize cloud services to perform a basic data analytics task, demonstrating the capabilities of cloud platforms for handling and processing data.
- ▶ Requirements:
 1. Data Source: Obtain a publicly available dataset (e.g., from Kaggle, UCI Machine Learning Repository, or a cloud provider's public dataset). Upload it to cloud object storage (e.g., S3, Azure Blob Storage, Cloud Storage).
 2. Processing Service: Choose a cloud data processing service:
 - Option A (Serverless/Managed Query): Use a serverless query engine (e.g., AWS Athena, Azure Synapse Analytics Serverless SQL pool, Google BigQuery) to query data directly from object storage using SQL.
 - Option B (Managed Cluster): Set up a small managed big data cluster (e.g., AWS EMR, Azure HDInsight, Google Dataproc) with a tool like Spark or Hive. Load the data and run a simple query or transformation.
 3. Analytics Task: Perform a simple analytics task (e.g., calculating averages, counts, filtering data, or basic aggregations).
 4. Output/Visualization (Optional): Store the results back into cloud storage or display them.
- ▶ Discussion Points:
 1. Compare the benefits of using a serverless query engine versus a managed cluster for data analytics.
 2. How do cloud services simplify the process of setting up and scaling big data infrastructure?
 3. Discuss the cost implications of different cloud data analytics approaches.

Problem 13.3: Compare Different Cloud Security Tools (Advanced/Hands-on)

- ▶ Description: Building on Week 12, this exercise requires a more in-depth comparison

or a hands-on exploration of specific cloud security tools, focusing on a particular security domain (e.g., network security, compliance, identity).

► Requirements:

1. Focus Area: Choose a specific cloud security domain (e.g., network security, identity and access management, security monitoring, data protection).
2. In-Depth Tool Comparison: Within your chosen domain, compare two or three tools/services from different cloud providers or a more detailed comparison of specific features within one provider's ecosystem.
3. Hands-on Exploration (If Possible): For at least one tool, attempt to configure and use a basic feature (e.g., set up a simple security group, define an IAM role, enable logging with CloudWatch/Azure Monitor/Cloud Logging).

► Discussion Points:

1. How do the chosen tools integrate with other services in the cloud ecosystem?
2. Discuss the management overhead and complexity associated with configuring and maintaining these security tools.
3. What are the latest trends or emerging technologies in cloud security for your chosen domain?

Problem 13.4: Write a Program to Implement User Authentication in a Cloud

Environment (Advanced Features)

► Description: Enhance the user authentication program from Week 12 by adding more advanced security features or integrating with a different aspect of cloud identity.

► Requirements:

1. Add Advanced Authentication Feature:
 - Implement Multi-Factor Authentication (MFA) for user login.
 - Implement Forgot Password / Password Reset functionality.
 - Integrate with social logins (e.g., Google, Facebook) using the cloud identity service.
 - Implement Role-Based Access Control (RBAC) where users can have different roles (e.g., admin, user) with different access levels to resources.
2. Program Functionality: Update your program to reflect the chosen advanced feature.
3. Deployment: Deploy the updated program to your chosen cloud environment.
4. Code & Demonstration: Provide the updated source code and demonstrate the new authentication flow.

► Discussion Points:

1. Discuss the security implications and user experience trade-offs of the advanced authentication feature you implemented.

2. How does RBAC improve the security posture and management of access in cloud applications? What are the challenges of managing user identities across multiple cloud environments or on- premises systems (identity federation)?

WEEK #14**Cloud Simulation with CloudSim**

Objective: To understand cloud resource management and scheduling by creating a simulation environment using CloudSim, and to analyze the impact of different policies.

Theory/Concepts

- ▶ Cloud Computing Simulation: Why simulation is used in cloud computing research and development (cost analysis, performance prediction, resource management strategy testing).
- ▶ CloudSim Architecture: Understanding key components: Datacenter, Host, VM, Cloudlet (tasks), Broker, DatacenterBroker.
- ▶ Resource Allocation Policies: How VMs are allocated to hosts.
- ▶ Scheduling Policies: How cloudlets (tasks) are scheduled on VMs (e.g., FCFS, Round-Robin, Space-shared, Time-shared).
- ▶ Load Balancing: Distributing workload evenly across resources.
- ▶ Resource Provisioning: Dynamic allocation and deallocation of computing resources based on demand.
- ▶ Performance Metrics in Simulation: Measuring metrics like makespan, throughput, response time, resource utilization, energy consumption.

Lab Setup

- ▶ Java Development Kit (JDK): Installed.
- ▶ CloudSim Library: Download and configure CloudSim (typically involves adding its JARs to your Java project's build path). You can find CloudSim on GitHub or its official website.
- ▶ IDE: An IDE like IntelliJ IDEA or Eclipse is highly recommended for Java development.

Lab Exercises (Unsolved Problems)**Problem 14.1: Create a Basic CloudSim Simulation Environment**

- ▶ Description: This exercise introduces you to CloudSim by setting up a minimal simulation environment.
 - ▶ Requirements:
1. Setup CloudSim Project: Create a new Java project in your IDE and add the CloudSim library to its dependencies.

2. Define Datacenter: Create at least one Datacenter entity with specified characteristics (e.g., number of hosts, MIPS of PEs, RAM, storage, bandwidth).
 3. Define Hosts: Within the datacenter, define at least two Host entities with specific CPU cores (PEs), RAM, and storage.
 4. Define Virtual Machines (VMs): Create several Vm objects with varying requirements (e.g., MIPS, RAM, image size, bandwidth).
 5. Define Cloudlets (Tasks): Create multiple Cloudlet objects representing computational tasks, specifying their length (MI - Million Instructions), file size, output size, etc.
 6. Create DatacenterBroker: Instantiate a DatacenterBroker to act on behalf of users, submitting VMs and Cloudlets.
 7. Simulation Execution: Configure and run the CloudSim simulation.
 8. Basic Output: Print basic simulation results (e.g., final simulation time, number of finished cloudlets).
- Discussion Points:
1. Explain the role of each CloudSim entity (Datacenter, Host, VM, Cloudlet, Broker).
 2. How do you define the resource characteristics for each entity?
 3. What are the advantages of using a simulation tool like CloudSim for cloud research?

Problem 14.2: Implement and Compare Resource Allocation and Scheduling Policies in

CloudSim

- Description: Implement and compare different resource allocation and scheduling policies within your CloudSim simulation, analyzing their impact on performance metrics.
- Requirements:
1. Extend Problem 14.1: Use the basic simulation environment created previously.
 2. Implement Policies:
 - Resource Allocation: Use different VmAllocationPolicy strategies (e.g., the default one, or a custom one if you choose to implement a simple VmAllocationPolicySimple).
 - Cloudlet Scheduling: Implement and compare:
 - First-Come-First-Serve (FCFS) Scheduling: Cloudlets are executed in the order they arrive. This is often the default behavior or easily achievable.
 - Round-Robin Scheduling: Cloudlets are given a time slice on a VM, and the VM switches between cloudlets. You may need to extend CloudSim's CloudletScheduler-TimeShared or implement your own.
 3. Experimentation: Run simulations with the same set of VMs and Cloudlets but using different policies.

4. Performance Analysis: Collect and analyze performance metrics such as:
 - Makespan: Total time to complete all cloudlets.
 - Average Cloudlet Finish Time: Average time for individual cloudlets to complete.
 - Resource Utilization: CPU, RAM, bandwidth utilization of hosts and VMs.
5. Reporting: Present your results in tables and graphs, comparing the performance under different policies.
 - ▶ Discussion Points:
 1. Describe how FCFS and Round-Robin scheduling work in a cloud context.
 2. Which policy performs better for your specific workload and why?
 3. How do different allocation policies affect overall resource utilization and VM placement?

Problem 14.3: Implement and Test Load Balancing and Resource Provisioning Strategies

in CloudSim

- ▶ Description: Implement and test basic load balancing and resource provisioning strategies in your CloudSim environment.
 - ▶ Requirements:
1. Extend Problem 14.2: Use an extended simulation environment with more dynamic aspects.
 2. Load Balancing:
 - Introduce a scenario where some hosts/VMs become overloaded while others are underutilized.
 - Implement a simple load balancing strategy (e.g., migrating a VM from an overloaded host to an underutilized one, or reassigning cloudlets). You might need to extend DatacenterBroker or VmAllocationPolicy.
 3. Resource Provisioning:
 - Simulate dynamic workload changes (e.g., cloudlets arriving over time, or a sudden burst of tasks).
 - Implement a basic resource provisioning strategy (e.g., adding/removing VMs or hosts based on a utilization threshold). This often involves modifying the DatacenterBroker to request/release VMs dynamically.
 4. Experimentation: Run simulations with and without your load balancing/provisioning strategies enabled.
 5. Performance Analysis: Analyze the impact of these strategies on:
 - Makespan and Throughput: Overall job completion and rate of completion.
 - Resource Utilization: How evenly resources are used.

- Cost (Optional): If you can integrate a simple cost model into CloudSim.

6. Reporting: Present results, comparing scenarios with and without your strategies.

► Discussion Points:

- 1.** Explain the importance of load balancing and resource provisioning in real-world cloud environments.
- 2.** Describe the algorithms you implemented for load balancing and resource provisioning.
- 3.** What are the challenges of implementing effective load balancing and provisioning in a dynamic cloud?

How could your strategies be improved (e.g., more sophisticated algorithms, predictive scaling)?